

Data Provenance

Origin, lineage, integrity and accountability across data, AI and content systems

From first principles and formal models to OpenLineage, cryptographic attestations, C2PA, the EU AI Act and the philosophy of trusted evidence.

PROGRAMME

01	**Foundations** Definitions and vocabulary	02	**Why it matters** Failures and value	03	**Theory and models** PROV, semirings
04	**Lineage engineering** OpenLineage in practice	05	**Capture techniques** Instrument and version	06	**Integrity and trust** Cryptographic evidence
07	**Machine learning and AI** Datasets to models	08	**Content provenance** C2PA and synthetic media	09	**Science** Reproducibility
10	**Regulation** GDPR, AI Act, BCBS 239	11	**Supply chains** Software and physical	12	**Governance** Roles, maturity, metrics
13	**Implementation** Roadmap and cases	14	**Deeper aspects** Epistemology and power	15	**Synthesis** Glossary and references

Gold numbers build the engineering core. Teal numbers apply it to domains. Dark tiles turn knowledge into action and reflection.

LEARNING OUTCOMES

By the end you will be able to do six things

Define precisely

Separate provenance, lineage, pedigree, audit trail and chain of custody, and explain why the differences matter.

Model formally

Express any process as entities, activities and agents in W3C PROV, and reason with why, where and how provenance.

Engineer capture

Design lineage collection with OpenLineage, column parsing, versioned tables and content addressed storage.

Secure evidence

Make provenance tamper evident with hashes, Merkle trees, signatures, transparency logs and attestations.

Satisfy obligations

Map GDPR, the EU AI Act, BCBS 239, SLSA and C2PA requirements to concrete provenance capabilities.

Think critically

Recognise what provenance cannot prove, where it creates privacy risk, and whose story it tells.

“

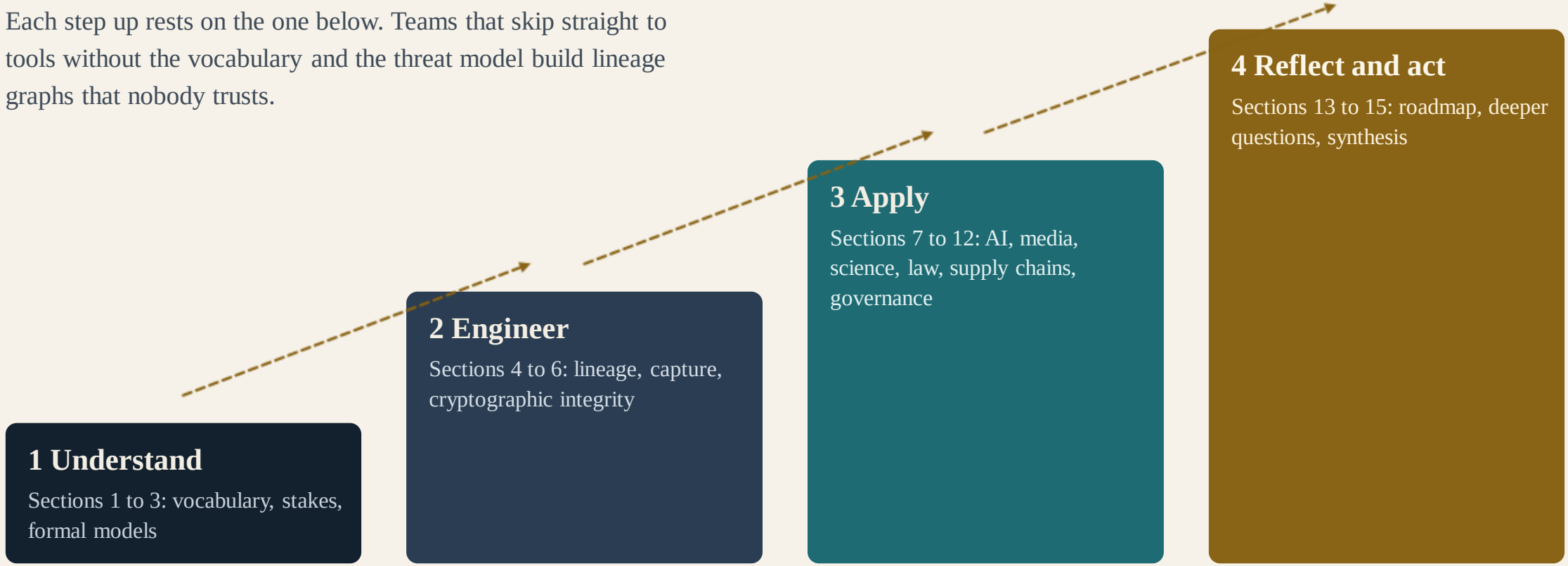
Every number has a past. Provenance is the discipline of remembering it, proving it, and being honest about what we cannot remember.

THE ORGANISING IDEA OF THIS MASTERCLASS

THE JOURNEY

Four layers, from concept to conscience

Each step up rests on the one below. Teams that skip straight to tools without the vocabulary and the threat model build lineage graphs that nobody trusts.



01

SECTION ONE

Foundations

What provenance is, where the idea came from, and the vocabulary we will use with precision for the rest of the masterclass.

- Definition, etymology and history
- Provenance versus lineage and its cousins
- Granularity, prospective and retrospective views
- Anatomy of a provenance record



A working definition, annotated

Data provenance is the documented, verifiable history of a piece of data: where it originated, which processes and agents created, transformed or moved it, when and under what conditions this happened, and with what rights and purposes attached.

ORIGIN

Source system, sensor, person or upstream dataset, plus collection method.

PROCESS

Every transformation, join, filter, model or manual edit applied.

AGENCY

The people, software and organisations responsible, and on whose behalf.

TIME

When each step occurred, and which version existed at that moment.

TERMS

Licence, consent, purpose limits, classification and retention.

The W3C frames provenance as information about the entities, activities and people involved in producing something, used to **assess its quality, reliability or trustworthiness**. The key word in our definition is *verifiable*: a history nobody can check is a story, not provenance.

The word came from art and archives before it reached data

LANGUAGE

To come forth

From the French *provenir*, rooted in the Latin *provenire*: to come forth, to originate.

ART

Chain of ownership

Dealers and museums document every owner of a painting to prove authenticity and lawful title.

ARCHIVES

Respect des fonds

Records stay grouped by their creator so their context and meaning survive.

COMPUTING

Derivation history

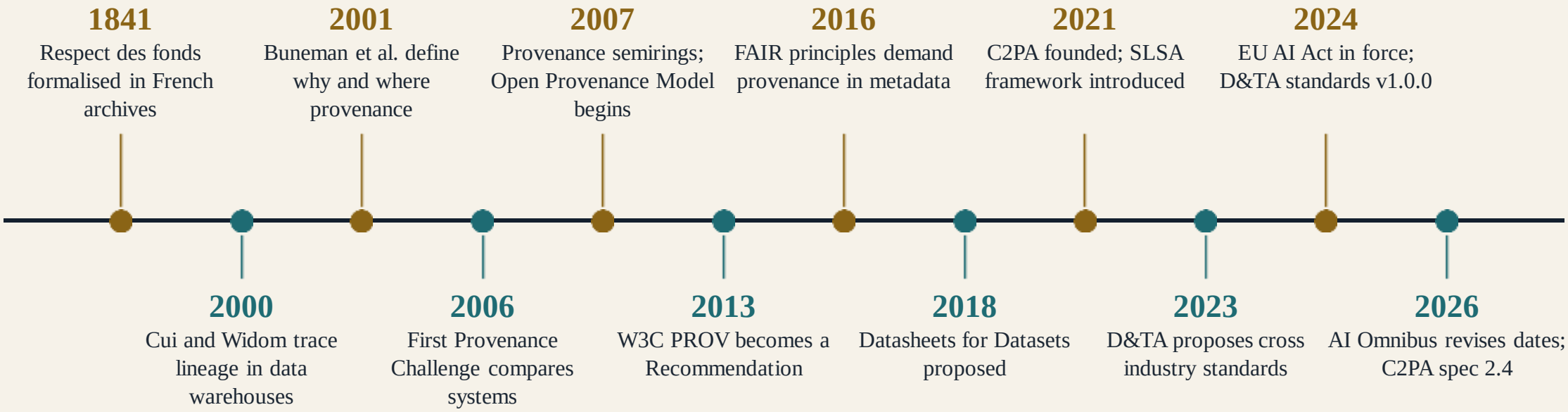
Databases, workflows and AI inherit both ideas: ownership and context.

Lesson from art: provenance gaps are red flags. A painting with no record for the years 1933 to 1945 demands scrutiny; so does a dataset with an unexplained gap in its history.

Lesson from archives: context is evidence. A record separated from its creator and purpose loses meaning, exactly like a column copied without its schema or definitions.

Two centuries of provenance in twelve milestones

Above the line: research and theory. Below the line: standards, industry and law.

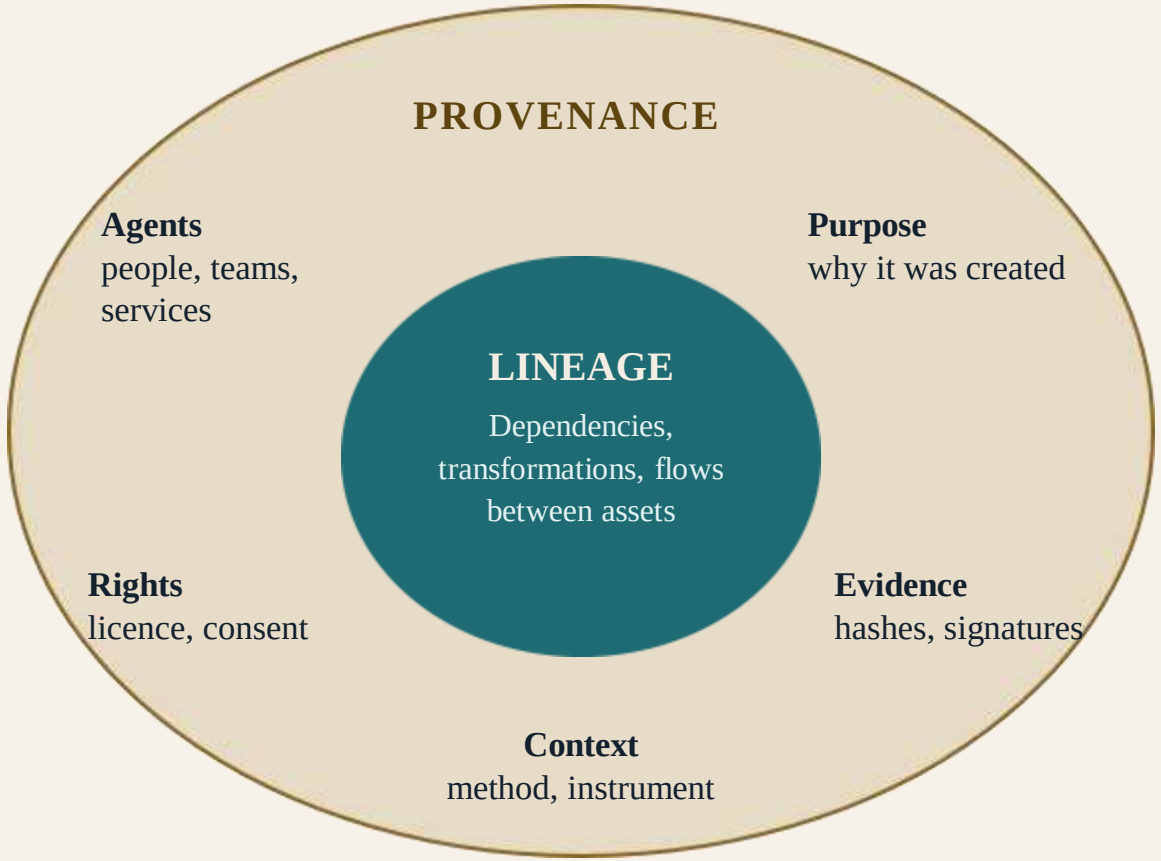


Provenance and its cousins are not synonyms

Term	Core question	Typical scope	Typical artefact
Provenance	Where did this come from, and can I trust it?	Origin, process, agents, time, rights	PROV graph, signed manifest
Lineage	What flowed into what?	Data dependencies between assets	Lineage graph in a catalog
Pedigree	How reliable are the ancestors?	Quality and authority of sources	Source ratings, intelligence grades
Audit trail	Who did what, and when?	Actions on a system or record	Append only event log
Chain of custody	Who held it, unbroken?	Possession and handling of evidence	Signed transfer forms, seals
Version history	What changed between states?	Successive states of one asset	Commits, snapshots
Metadata	What is this, structurally?	Descriptive and technical attributes	Schema, data dictionary

Rule of thumb: lineage is the skeleton, provenance is the full body. Lineage tells you the edges; provenance adds who, why, under which terms, and the evidence that the edges are real.

Lineage sits inside provenance



Lineage answers

If table A changes, which dashboards break? Which job produced this column?

Provenance also answers

Were we allowed to use A for this purpose? Who approved the change? Can we prove the job ran on the declared inputs?

A lineage graph with no agents, purposes or evidence is necessary but not sufficient for trust.

The seven questions every provenance record must answer

What

Identity of the asset: stable ID, version, content hash, schema.

Where

Source systems, locations, jurisdictions and storage.

When

Event time, processing time, validity period.

Who

Creators, operators, approvers and the party they acted for.

How

Code version, parameters, method, instrument, model.

Why

Purpose of collection and of each processing step.

Terms

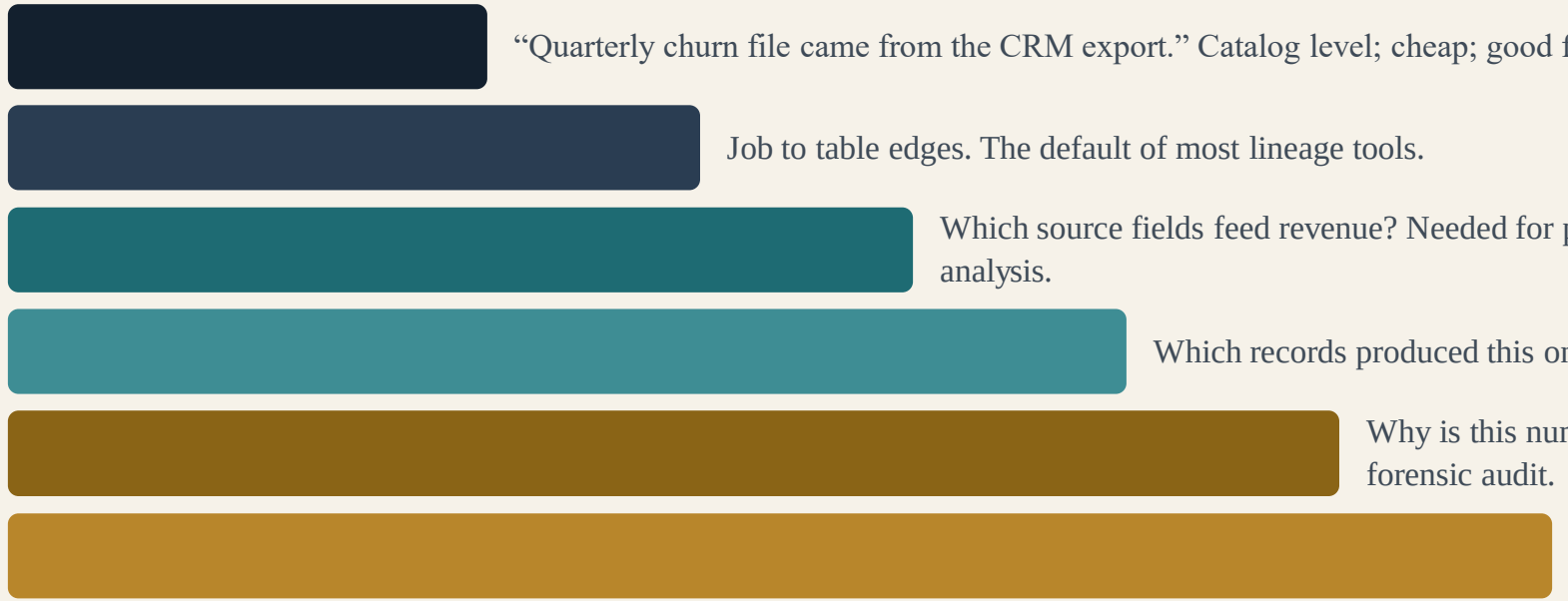
Licence, consent basis, restrictions, retention.

Plus one meta question

How do we know this record itself is true?

Use the seven questions as a design checklist for any capture system and as an interview script when auditing an existing dataset.

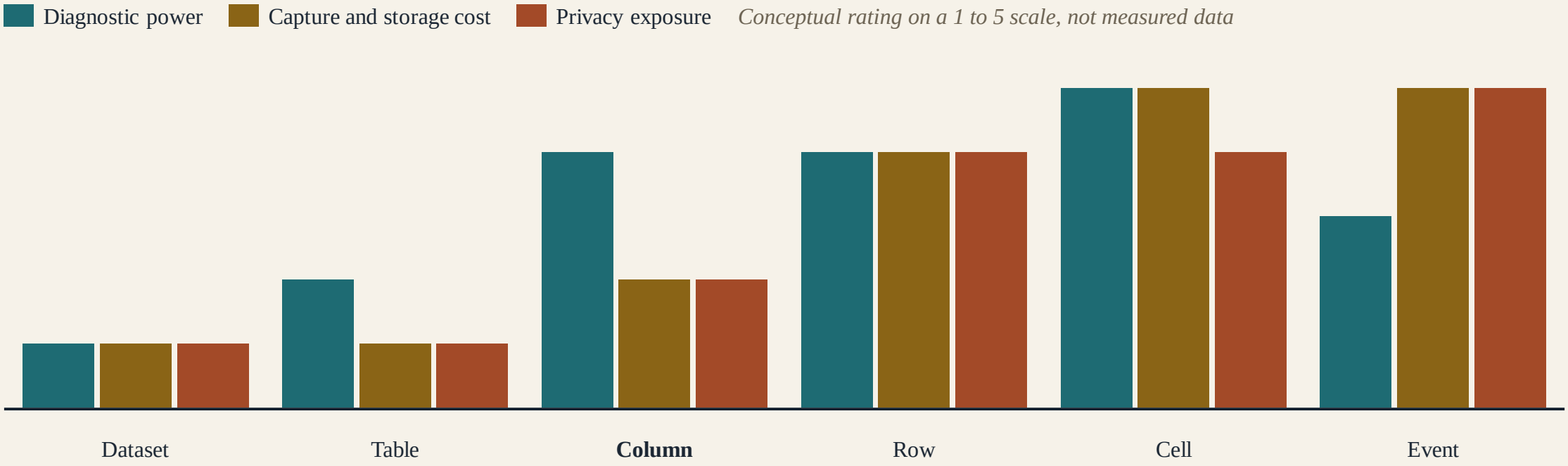
Granularity: the ladder from coarse to fine



Dataset	“Quarterly churn file came from the CRM export.” Catalog level; cheap; good for inventories.
Table or file	Job to table edges. The default of most lineage tools.
Column	Which source fields feed revenue? Needed for privacy and impact analysis.
Row or record	Which records produced this one? Erasure, recalls.
Cell or value	Why is this number 42? Database theory, forensic audit.
Event or byte	System calls, every read.

Bar length suggests how quickly the volume of provenance grows as you descend the ladder. **Choose the coarsest level that still answers your questions**, and descend only where risk justifies it.

The trade off: diagnostic power against capture cost



Column level is often the sweet spot: a large jump in power for a moderate rise in cost. At event level, power can fall because the **semantic gap** hides business meaning under raw system activity.

Recipe, execution and evolution: three kinds of history

PROSPECTIVE

The recipe

extract → clean → train

Workflow definitions, DAG code, SQL models. Describes what *should* happen.

RETROSPECTIVE

The execution record

run 77 → run 78 → run 79

Actual inputs, versions, parameters, timings and outputs. Records what *did* happen.

EVOLUTION

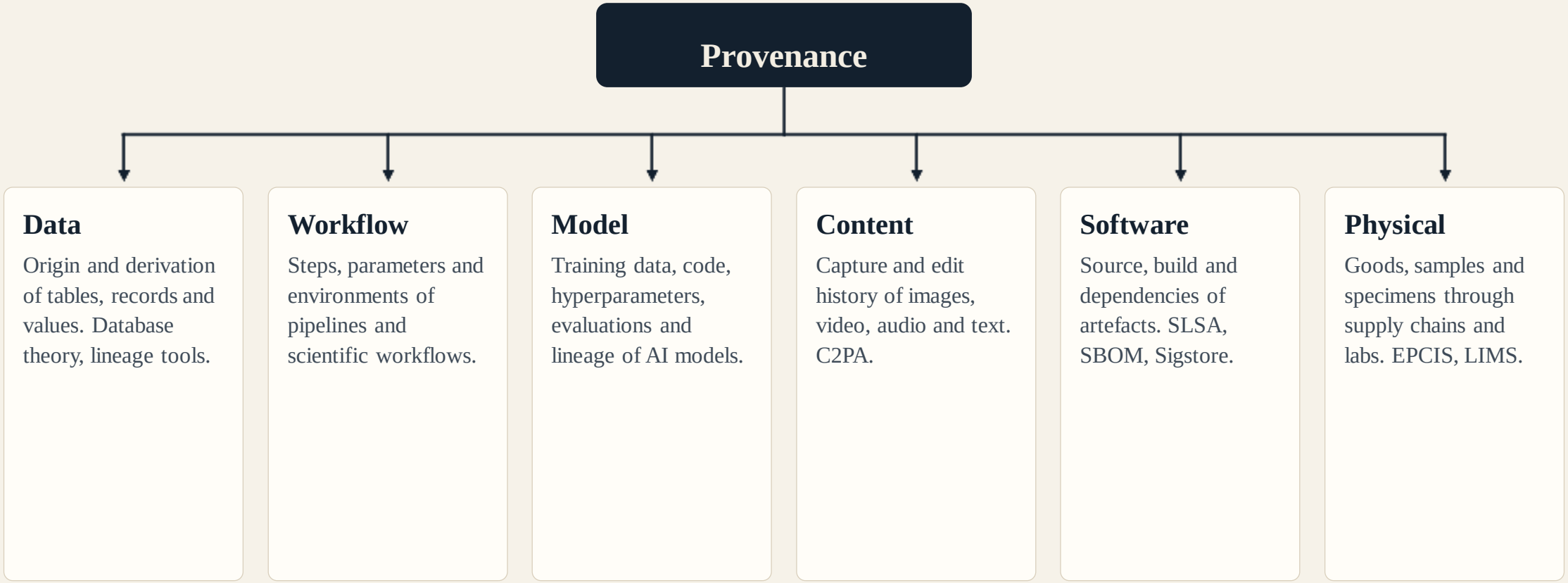
The recipe's history

v1 → v2 → v3

How the pipeline itself changed: who edited the logic, when, and why.

⚠ The most common audit failure is presenting the recipe as if it were the execution record. A DAG shows what was intended; only retrospective provenance proves what ran, on which inputs, with which code.

A taxonomy of provenance domains



Six people, six different needs from the same history



Data engineer

What breaks if I change this table?



Data scientist

Can I reproduce last quarter’s model?



Auditor

Show me evidence this figure is right.



Privacy officer

Where does this person’s data flow?



Executive

Can I sign this report with confidence?



Citizen

Is this image real? Who decided about me?

Design implication: one provenance store, many views. Engineers want graphs, auditors want evidence packs, citizens want a simple badge they can trust.

Portraits are illustrative stock photographs, not real practitioners.

Structure of a provenance record

```
{
"asset": "sales_monthly v42",
"content_hash": "sha256:9f2c…e71a",
"generated_by": "run 8812 of job revenue_rollup",
"used": ["orders v311", "fx_rates v97"],
"agent": "svc_etl for Finance Data Team",
"code": "git commit 4b1e0d7",
"time": "06 Nov 2026, 09:00 UTC",
"purpose": "statutory monthly reporting",
"licence": "internal, confidential",
"signature": "ed25519:7a1d…"
}
```



- **Identity:** stable name, version and content hash
- **Derivation:** the activity and its exact inputs
- **Agency:** software actor and accountable owner
- **Reproducibility:** immutable code reference
- **Time:** unambiguous, time zone explicit
- **Terms:** purpose and licence travel with data
- **Integrity:** signature binds every field above

SECTION 01 RECAP

Six ideas to carry forward

1. Provenance is a **verifiable** history of origin, process, agency, time and terms.
2. Lineage is the dependency skeleton inside provenance, not a synonym for it.
3. Art and archives teach us that gaps are red flags and context is evidence.
4. Granularity is the master design choice; column level is often the sweet spot.
5. The recipe is not the execution record. Audits need retrospective evidence.
6. One history serves many personas; design the views, not just the store.

Next: why any of this is worth the effort.

02

SECTION TWO

Why provenance matters

Real incidents where missing history cost money, credibility and lives, and the value that good provenance unlocks.

Failure modes when history is missing

Five case studies from economics to AI

Impact analysis and root cause tracing

The economics of provenance



Trust in data is earned from three kinds of evidence

Trust = *Origin* × *Process* × *Accountability*

Origin clarity

Do we know the source, its method and its authority? Sensor calibration, survey design, licence.

Process transparency

Can we inspect every transformation, and rerun it to the same result?

Accountability

Is a named person or team answerable for each step and each decision?

The factors **multiply** rather than add: a perfect pipeline fed by an unknown source still yields untrustworthy output. Any factor near zero collapses the whole product.

Six failure modes of organisations without provenance

Silent drift

An upstream schema or definition changes and nobody downstream is told. Reports stay green while meaning rots.

Stale truth

A dashboard silently reads yesterday's snapshot because a job failed. No record shows which version fed it.

Duplicated logic

Three teams compute "active customer" three ways. Without lineage, nobody can find or reconcile them.

Rights violations

Data collected for one purpose leaks into another, or licensed data trains a model that the licence forbids.

Irreproducibility

A model or published result cannot be recreated because inputs, code or environment were never pinned.

Untraceable breach

After an incident, nobody can say which copies of leaked data exist, so notification scope is guesswork.

Every failure mode shares a root: **the question "where did this come from?" has no fast, trustworthy answer.**

02 · CASE STUDY 1 · ECONOMICS

A spreadsheet range that shaped austerity debates

What happened

A 2010 paper by Reinhart and Rogoff, “Growth in a Time of Debt”, linked public debt above 90 percent of GDP with sharply lower growth and was widely cited in policy debates. In 2013 Herndon, Ash and Pollin tried to replicate it from the original spreadsheet.

What they found

An averaging formula omitted five countries, some years of data were excluded, and an unusual weighting was applied. Corrected, mean growth in the high debt group was about 2.2 percent rather than slightly negative.

Provenance lesson

Cell level derivation was invisible. Recorded formula lineage and published inputs would have exposed the omission immediately.

5

countries dropped by one averaging range

THE FORMULA, SIMPLIFIED

AVERAGE(L30 : L44)

should have covered L30 : L49

02 · CASE STUDY 2 · PUBLIC HEALTH

Covid test results lost to a file format limit

15,841

positive cases in England missed from daily reporting, late September to early October 2020

65,536

maximum rows in the legacy XLS spreadsheet format used in the pipeline

What happened

Laboratory results were combined through a process that used the old XLS format. When files exceeded its row limit, records were silently truncated, so cases were neither reported nor passed promptly to contact tracing.

Why it went unnoticed

No reconciliation compared records received with records loaded. The pipeline had no row level provenance count at each hop.

Provenance lesson

Record input and output counts per run, alert when they diverge, and treat every manual file hop as a lineage edge that must be observed.

02 · CASE STUDY 3 · GENOMICS

When software quietly rewrote gene names as dates

RESEARCHER TYPES

SEPT2

a human gene symbol



auto format

FILE STORES

2 Sep

a calendar date

1 in 5

papers with Excel gene lists were affected, in a 2016 survey by Ziemann, Eren and El Osta

The provenance failure

A tool applied an undocumented transformation. The output carried no record that conversion had happened, so supplementary data in published papers silently diverged from the researchers' intent.

The outcome

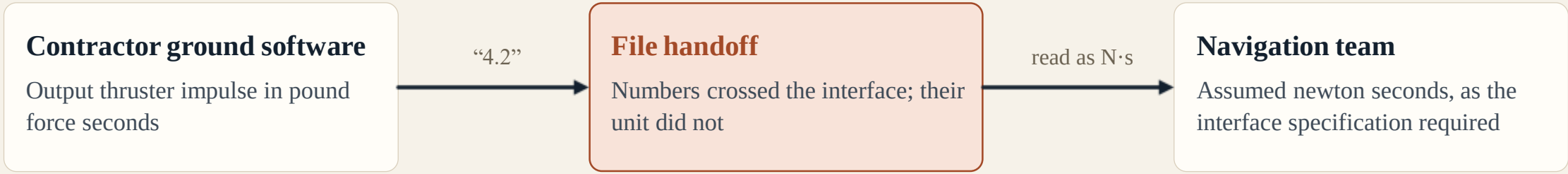
In 2020 the HUGO Gene Nomenclature Committee renamed affected genes, for example SEPT1 to SEPTIN1 and MARCH1 to MARCHF1. The world changed its vocabulary to cope with invisible transformations.

The lesson

Every tool in the chain is an agent. Implicit transformations are activities and must be captured, or prevented with typed, schema aware formats.

02 · CASE STUDY 4 · AEROSPACE

Mars Climate Orbiter: data without its units



4.45

newton seconds in one pound force second: the size of the silent error

On 23 September 1999 the spacecraft entered the Martian atmosphere far lower than planned and was lost. The investigation board identified the unit mismatch in trajectory correction data as the root cause.

Provenance lesson: semantic context such as units, reference frames, currencies and time zones is part of a value’s provenance. Strip it at an interface and the number becomes a different fact.

02 · CASE STUDY 5 · ARTIFICIAL INTELLIGENCE

Web scale training data and the provenance reckoning

CONTENT SAFETY

LAION 5B, December 2023

Researchers at the Stanford Internet Observatory reported that this public image and text link dataset contained links to illegal child abuse imagery. The maintainers withdrew it and later released a cleaned version. Billions of links had been gathered with no per item provenance review.

COPYRIGHT

Litigation over sources

Publishers and authors have sued AI developers over training data, including The New York Times against OpenAI and Microsoft in December 2023. In 2025 a class action by authors against Anthropic was settled for a reported 1.5 billion US dollars, with the origin of book copies central to the case.

LICENSING

Laundered licences

The Data Provenance Initiative audited more than 1,800 fine tuning datasets in 2023 and found that licence information on popular hosting sites was frequently missing or wrong. Restrictive terms disappear as datasets are repackaged.

Provenance lesson: for AI, provenance is no longer a quality nicety. It decides whether a model is lawful to train, safe to release, and defensible in court. Section seven covers the tooling.

The cost of not knowing is large, if hard to measure

PER ORGANISATION

$12.9M

average yearly cost of poor data quality to organisations, as estimated by Gartner research published in 2021

PER ECONOMY

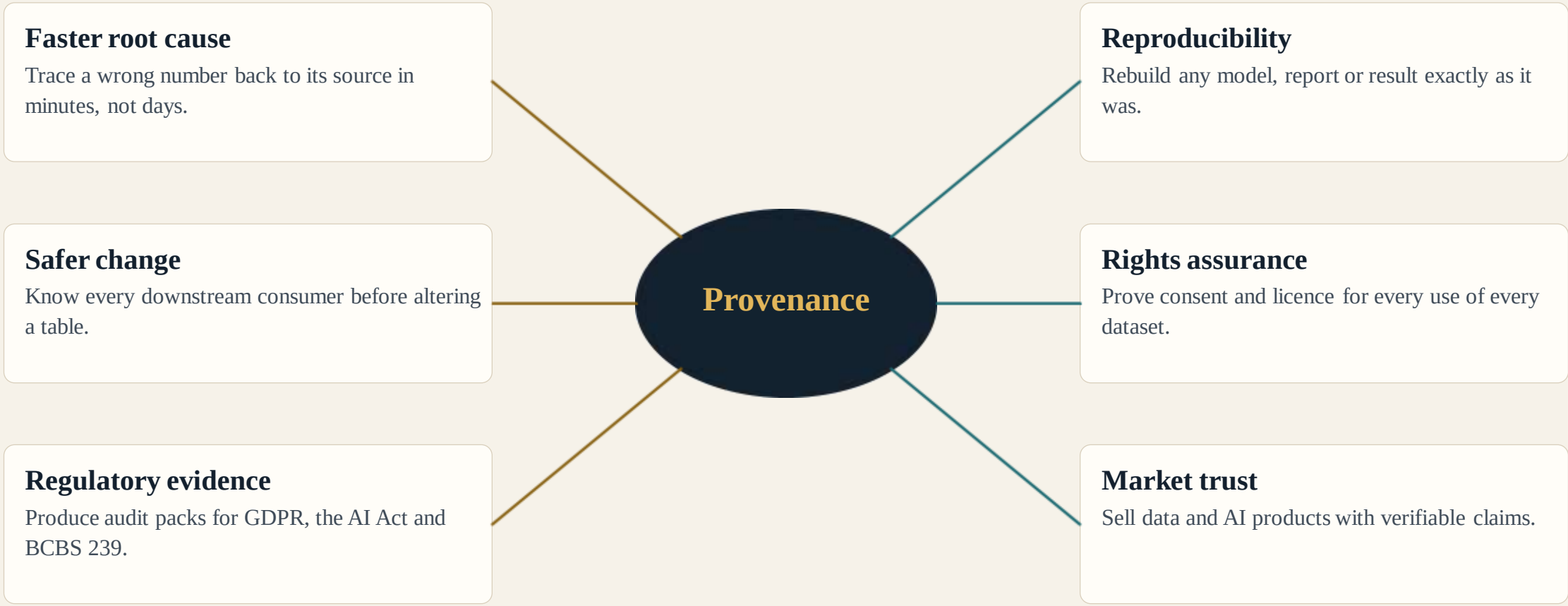
$3.1T

yearly cost of bad data to the US economy, an IBM estimate cited in Harvard Business Review in 2016

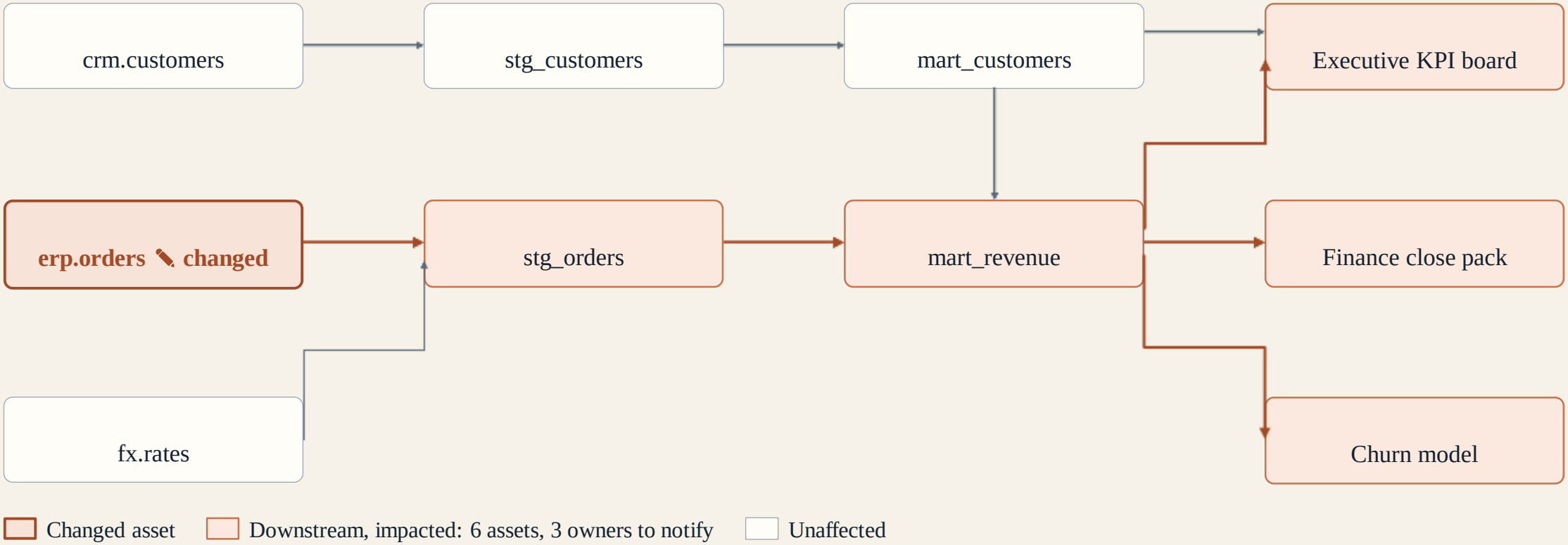
Read these with care. Both are broad estimates with contested methods. Their value is in signalling scale, not in budgeting.

Measure your own. Track hours spent tracing incidents, rework from wrong figures and audit findings. These become the baseline for a provenance business case.

Six value levers radiate from one capability



Impact analysis: the blast radius of one change



Root cause: walking the graph backwards

1. **Symptom:** Executive board shows European revenue down sharply overnight.
2. **Upstream hop:** mart_revenue run 912 used fx.rates version 98, not 97.
3. **Version diff:** version 98 was loaded from a new vendor feed by job fx_ingest.
4. **Root cause:** the new feed quotes rates inverted, EUR per USD instead of USD per EUR.

WHAT MADE IT FAST

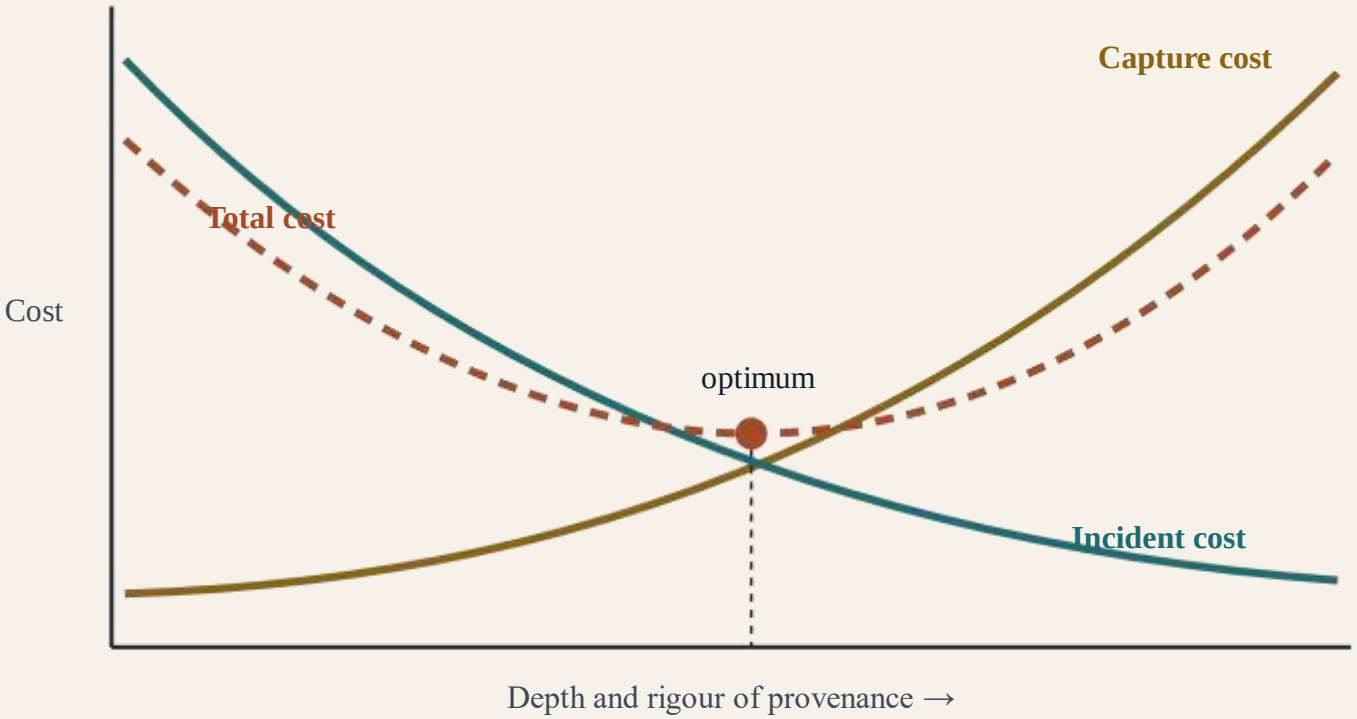
Run level lineage linking each output to exact input versions; version history of fx.rates; job metadata naming the new source.

WITHOUT PROVENANCE

Analysts inspect SQL by hand, interview teams and compare spreadsheets, often for days.

Illustrative scenario. Asset names and versions are invented for teaching.

The economics: an optimum, not a maximum



Conceptual curves for reasoning, not measured data.

Capture cost rises with depth: storage, compute and engineering time. Incident cost falls as diagnosis gets faster and fewer errors escape.

The optimum moves right as **regulatory stakes** rise, and moves right again as **automation** lowers capture cost.

Different assets deserve different points on the curve. Tier your data by risk.

SECTION 02 RECAP

The case for provenance in five sentences

1. Trust multiplies origin, process and accountability; one weak factor sinks the rest.
2. Famous failures in economics, health, genomics and aerospace were provenance failures at heart.
3. In AI, provenance decides legality, safety and defensibility, not just quality.
4. Forward traversal gives impact analysis; backward traversal with versions gives root cause.
5. Invest to an optimum set by risk tier, and let automation move that optimum.

03

SECTION THREE

Theory and models

The formal foundations: the W3C PROV data model and the database theory of why, where and how provenance.

Entities, activities and agents

PROV N and PROV O in practice

Why, where, how and why not provenance

Provenance semirings and graph constraints



The W3C PROV family: four standards, many companions

RECOMMENDATION

PROV DM

The conceptual data model: types, relations, structure.

RECOMMENDATION

PROV O

An OWL2 ontology for linked data and RDF stores.

RECOMMENDATION

PROV N

A compact human readable notation for examples and specs.

RECOMMENDATION

PROV CONSTRAINTS

Rules that define a valid, consistent provenance document.

NOTE

Overview and Primer

Entry points and gentle introduction with examples.

NOTE

PROV XML

An XML schema serialisation of the model.

NOTE

PROV AQ

How to find and retrieve provenance over the web.

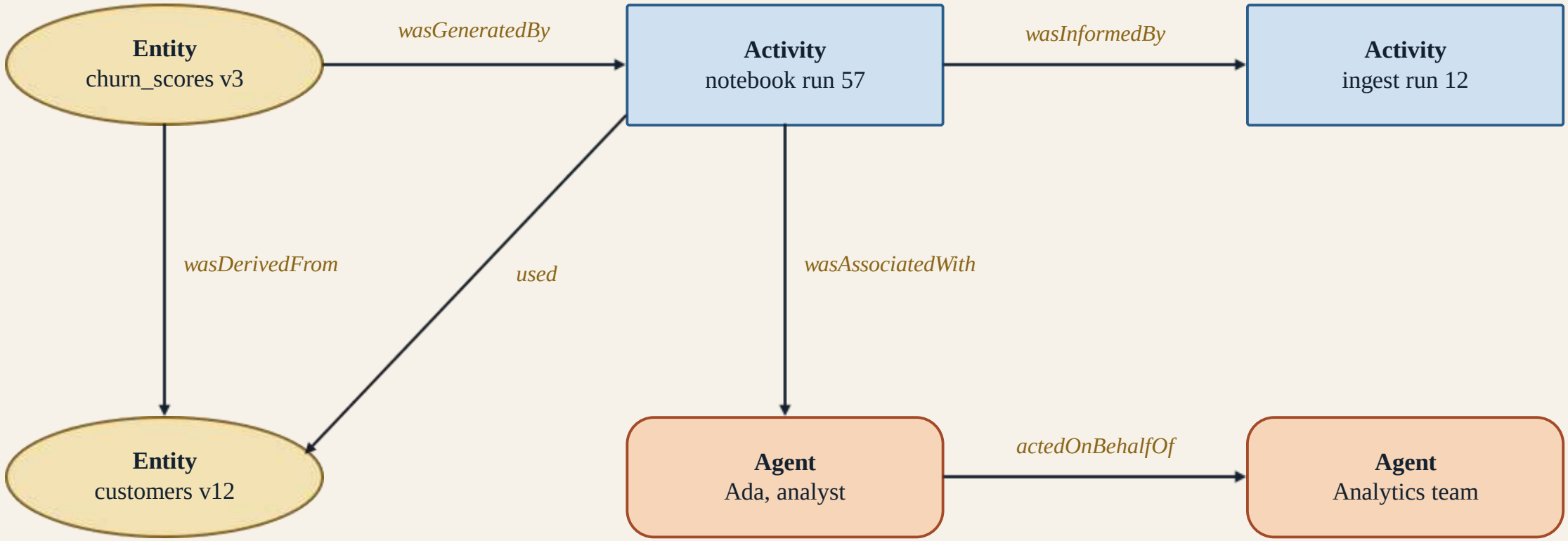
NOTES

Dictionary, DC, SEM, LINKS

Collections, Dublin Core mapping, formal semantics, bundle linking.

Published together on **30 April 2013** by the W3C Provenance Working Group. A JSON serialisation, PROV JSON, was later submitted as a W3C Member Submission and is widely used by libraries such as the Python *prov* package.

The PROV core: entities, activities, agents



Arrows point **backwards in time**, from the newer thing to what it depends on. Entities are things, activities happen over time, agents bear responsibility.

The PROV relations at a glance

Relation	From → To	Meaning in plain words
wasGeneratedBy	Entity → Activity	This thing came into existence through that activity
used	Activity → Entity	The activity consumed or read that thing
wasDerivedFrom	Entity → Entity	Content of one thing was transformed from another
wasAttributedTo	Entity → Agent	Someone is responsible for this thing
wasAssociatedWith	Activity → Agent	Someone was responsible for the activity
actedOnBehalfOf	Agent → Agent	Delegation: who acted for whom
wasInformedBy	Activity → Activity	One activity used something another generated
wasStartedBy / wasEndedBy	Activity → Entity	A trigger started or ended the activity
wasInvalidatedBy	Entity → Activity	The thing stopped being usable, for example deleted

All are specialisations of **wasInfluencedBy**, the generic “this affected that” relation, which is useful when you know of a dependency but not its kind.

From a sentence to a provenance graph

> “Ada, working for the Analytics team, ran notebook run 57, which read customers v12 and events v40 and produced churn_scores v3, owned by Analytics.”

Subject	Relation	Object
notebook run 57	used	customers v12
notebook run 57	used	events v40
churn_scores v3	wasGeneratedBy	notebook run 57
churn_scores v3	wasDerivedFrom	customers v12
notebook run 57	wasAssociatedWith	Ada
Ada	actedOnBehalfOf	Analytics team
churn_scores v3	wasAttributedTo	Analytics team

Method: nouns that exist become entities, verbs that take time become activities, and people or systems that are answerable become agents.

Judgement call: did the scores derive from events v40 too? Only if its content shaped them. Derivation is a semantic claim, not a mechanical one.

The same example in PROV N

```
document
prefix ex <http://example.org/>
entity(ex:customers_v12)
entity(ex:events_v40)
entity(ex:churn_scores_v3)
activity(ex:notebook_run_57)
agent(ex:ada, [prov:type='prov:Person'])
agent(ex:analytics, [prov:type='prov:Organization'])
used(ex:notebook_run_57, ex:customers_v12)
used(ex:notebook_run_57, ex:events_v40)
wasGeneratedBy(ex:churn_scores_v3, ex:notebook_run_57)
wasDerivedFrom(ex:churn_scores_v3, ex:customers_v12)
wasAssociatedWith(ex:notebook_run_57, ex:ada)
actedOnBehalfOf(ex:ada, ex:analytics)
wasAttributedTo(ex:churn_scores_v3, ex:analytics)
endDocument
```

Declarations first

Each node is typed once, with optional attributes in square brackets.

Relations as functions

Subject first, object second, then optional time and attributes.

Where it is used

Specifications, teaching, test fixtures, and conversion tools such as ProvToolbox.

PROV O in Turtle, and a query that walks the graph

TURTLE

```
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix ex: <http://example.org/> .

ex:churn_scores_v3 a prov:Entity ;
prov:wasGeneratedBy ex:notebook_run_57 ;
prov:wasDerivedFrom ex:customers_v12 ;
prov:wasAttributedTo ex:analytics .

ex:notebook_run_57 a prov:Activity ;
prov:used ex:customers_v12 , ex:events_v40 ;
prov:wasAssociatedWith ex:ada .

ex:ada a prov:Person ;
prov:actedOnBehalfOf ex:analytics .
```

SPARQL: ALL ANCESTORS

```
SELECT ?source WHERE {
ex:churn_scores_v3
prov:wasDerivedFrom+ ?source .
}
```

The + is a property path: follow derivation one or more hops. Transitive lineage in one line.

Because PROV O is an OWL ontology, it mixes freely with DCAT, schema.org and domain vocabularies in one knowledge graph.

Beyond the core: the expressive features of PROV

Qualified relations

Turn an edge into a node so it can carry time, role and attributes: a *Usage* at 09:00 in the role “training input”.

Plans

An association can name the *Plan* an agent followed: the script, SOP or protocol. This links retrospective to prospective provenance.

Roles

The function an entity or agent played: reviewer versus author, reference data versus input.

Bundles

A named set of provenance statements that is itself an entity, so provenance can have provenance.

Specialisation and alternates

“Version 3 of the report” specialises “the report”; two copies in different systems are alternates of one thing.

Collections

Entities that contain members, with *hadMember*; dictionaries add keyed insertion and removal.

Derivation subtypes

Revision, quotation and primary source: three different stories about how content moved.

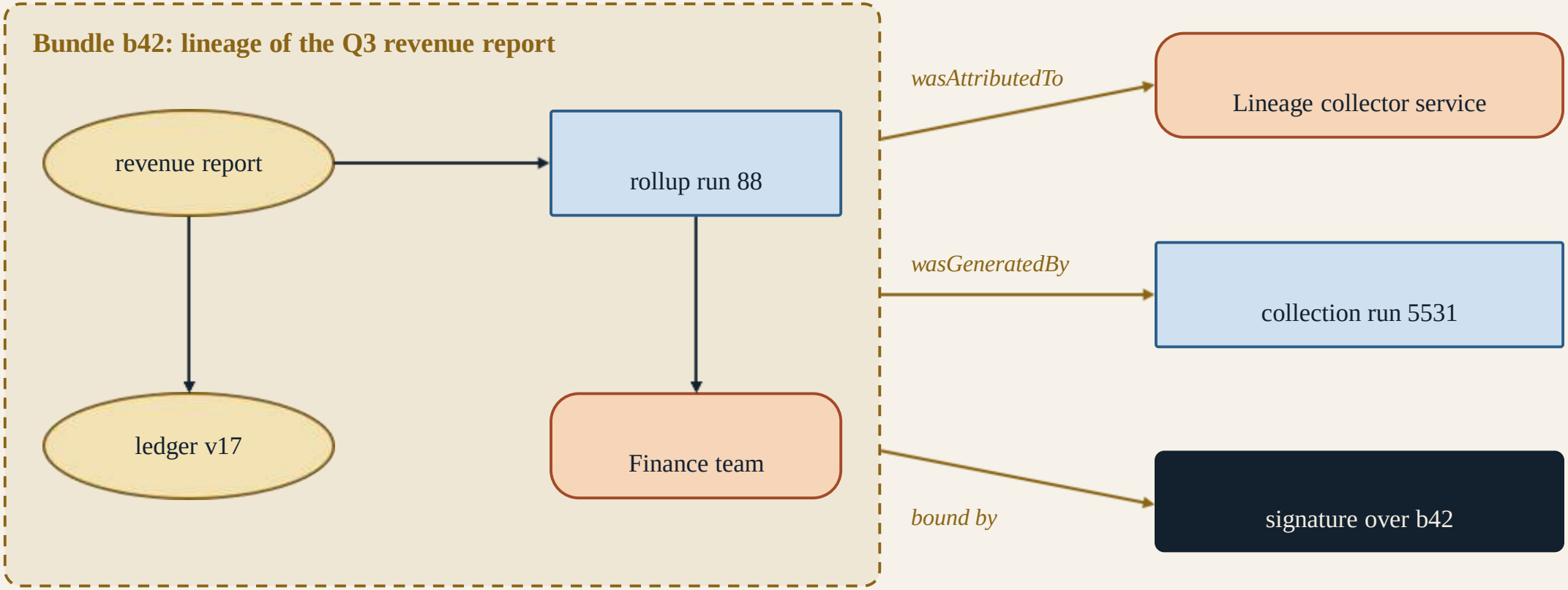
Time and location

Instants on generation, usage, start and end; *atLocation* for where things happened.

Extensibility

Subclass anything for your domain. ProvONE, PAV and many others do exactly this.

Bundles: the provenance of provenance



The dashed region is a bundle, an entity in its own right. Statements **about** it record who collected the lineage, when, and how it is protected, so consumers can judge the record before trusting the report.

From the Provenance Challenges to PROV

2006 to 2010
Three Provenance Challenges ask research systems to answer the same queries over a brain imaging workflow.

→

2007 to 2010
The Open Provenance Model emerges to make those systems interoperable.

→

2009 to 2010
W3C Provenance Incubator Group surveys needs across the web.

→

2011 to 2013
Provenance Working Group delivers the PROV family.

Open Provenance Model	W3C PROV	What changed
Artifact	Entity	Broadened beyond digital artefacts
Process	Activity	Explicit start and end times
wasControlledBy	wasAssociatedWith	Adds plans and delegation
wasTriggeredBy	wasInformedBy	Clearer communication semantics
Accounts	Bundles	Provenance of provenance made first class

Three questions database theory asks of every output tuple

Why?

Which input tuples justify this output existing?

Answer: sets of witnesses. Buneman, Khanna and Tan, 2001.

Where?

From which input location was this value copied?

Answer: source cells. Also Buneman, Khanna and Tan, 2001.

How?

In what combinations were inputs used to derive it?

Answer: polynomials. Green, Karvounarakis and Tannen, 2007.

Fine grained by design. These models work at tuple and cell level, inside the query engine, where table level lineage tools cannot see.

Why it matters to practitioners. Deletion propagation, access control on derived data and debugging wrong query results all reduce to these questions.

Why, where and how on one small join

Table R (employee, dept)

id	emp	dept
r1	Ann	Sales
r2	Bob	Sales
r3	Cy	Ops

Table S (dept, city)

id	dept	city
s1	Sales	Leeds
s2	Ops	Hull

```
SELECT DISTINCT city FROM R JOIN S ON R.dept = S.dept
```

Output	Why: witnesses	Where: copied from	How: polynomial
Leeds	{ r1, s1 } or { r2, s1 }	S.city in s1	r1·s1 + r2·s1
Hull	{ r3, s2 }	S.city in s2	r3·s2

Why lists alternative justifications. Remove r1 and Leeds survives through r2.

Where pinpoints the cell to fix if “Leeds” is misspelt.

How records joins as products and alternatives as sums.

One algebra, many meanings: provenance semirings

Annotate every input tuple with a value. Joins combine annotations with ⊗, unions and projections with ⊕. Choose the semiring, and the same query computes a different property.

Semiring	Values	⊕	⊗	What the annotation tells you
Polynomials N[X]	Polynomials	+	·	Full how provenance; the universal case
Boolean	true, false	or	and	Does the output exist? Set semantics
Counting	Natural numbers	+	×	How many copies? Bag semantics
Security	Clearance levels	min	max	Lowest clearance needed to see the output
Tropical	Costs	min	+	Cheapest derivation path
Viterbi	Probabilities	max	×	Most likely derivation
Why	Sets of sets	∪	pairwise ∪	Witness sets, as on the previous slide

The key theorem: every such semiring is a homomorphic image of N[X]. Compute the polynomial once, then evaluate it under any semiring later, without rerunning the query.

Evaluating “Leeds” = r1·s1 + r2·s1 five ways

Counting

r1 = 2, r2 = 3, s1 = 1

2×1 + 3×1

5

copies in a bag result

Boolean

r1 = T, r2 = F, s1 = T

(T and T) or (F and T)

true

Leeds is in the answer

Security

r1 = Secret, r2 = Public, s1 = Public

min(max(S,P), max(P,P))

Public

visible without clearance

Tropical

costs r1 = 4, r2 = 7, s1 = 2

min(4+2, 7+2)

6

cheapest derivation

Deletion

delete r1: set r1 = 0

0·s1 + r2·s1

r2·s1

Leeds survives via Bob

Security insight: although Ann’s record is Secret, the city is derivable from public data alone, so it may be released. Naive “taint” tracking would wrongly classify it Secret.

Erasure insight: after a deletion request for r1, provenance proves which outputs must change and which remain valid. That is GDPR erasure reasoning made exact.

The wider research landscape

2000

Warehouse lineage

Cui and Widom trace which source tuples produced a view tuple, through relational and general transformations.

2009

Why not provenance

Chapman and Jagadish explain why an *expected* answer is missing: which operator eliminated it.

2006 onwards

Uncertain data

Stanford's Trio system combined lineage with uncertainty, so confidence flows with derivation.

Engines

Provenance aware databases

Perm and GProM rewrite queries to compute provenance; ProvSQL brings semirings to PostgreSQL.

Big data

Lineage in dataflows

Research systems such as Titian add record level lineage to Spark for interactive debugging.

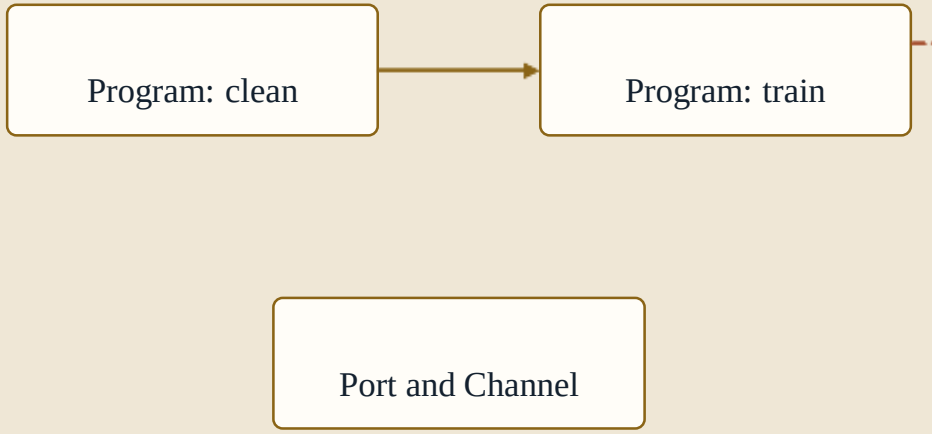
Frontier

Explanations and causality

Which inputs are most responsible for an output? Links provenance to causal inference and attribution in ML.

ProvONE: linking the plan to its executions

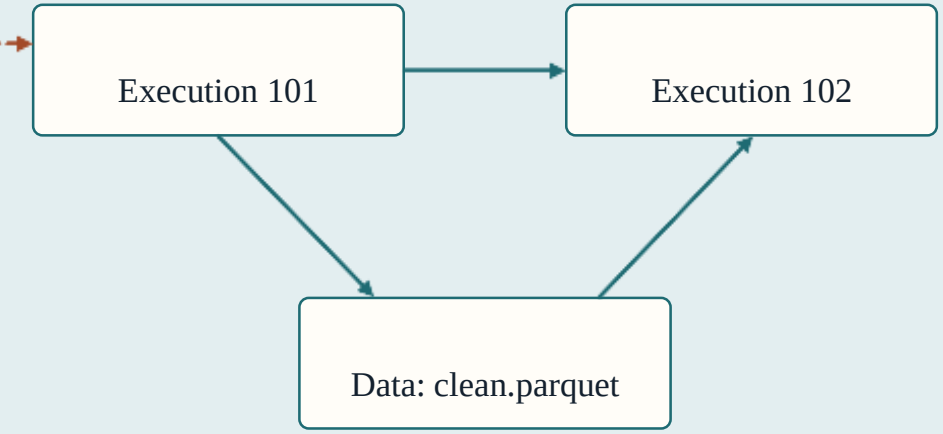
PROSPECTIVE: THE WORKFLOW



Programs, ports and channels describe what the workflow is designed to do.

hadPlan

RETROSPECTIVE: A RUN



Executions and data record what one run actually did, as PROV activities and entities.

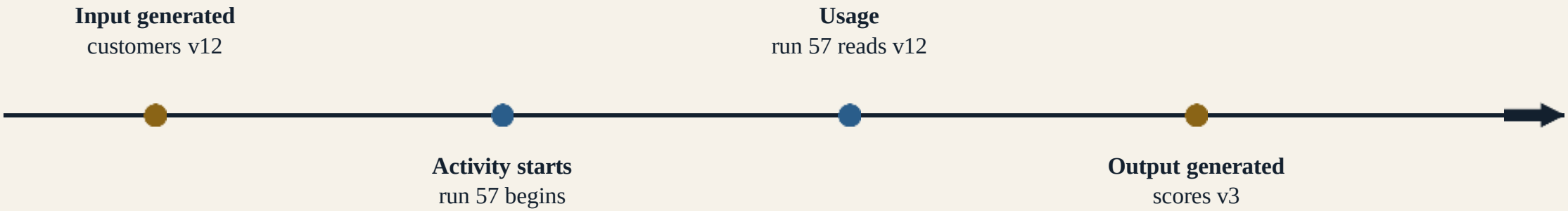
Workflow systems that pioneered this: **Kepler**, **Taverna**, **VisTrails**, **Pegasus** and **Galaxy**. For plain Python scripts, **noWorkflow** captures provenance without a workflow engine.

Companion vocabularies that complete the picture

Vocabulary	Steward	What it adds to provenance
DCAT 3	W3C	Describes datasets and distributions in catalogues; links them to PROV activities
Dublin Core	DCMI	Simple terms such as source, creator and provenance statement
ODRL	W3C	Machine readable permissions, prohibitions and duties for licences
DPV	W3C community group	Purposes, legal bases and processing categories for personal data
DQV	W3C	Quality measurements and annotations attached to datasets
PAV	Research community	Authoring and versioning: curated by, retrieved from, version
schema.org	Community	Web scale terms such as isBasedOn and sdPublisher
Croissant	MLCommons	Machine learning dataset metadata, including responsible AI fields

Design pattern: PROV for the “how it came to be”, ODRL and DPV for the “what you may do with it”, DCAT and Croissant for the “what it is”.

A valid provenance graph obeys the arrow of time



- **Generation precedes usage.** Nothing can be read before it exists.
- **Unique generation.** An entity is generated by at most one activity.
- **Invalidation is final.** No usage after an entity is invalidated.
- **Usage within the activity.** Reads happen between its start and end.
- **Derivation respects order.** A source is generated before what derives from it.
- **Violations are signals.** Clock skew, forged records or mislabelled versions.

SECTION 03 RECAP

The theory in six statements

1. PROV models any history as entities, activities and agents joined by typed, backward pointing relations.
2. Usage is a fact; derivation is a semantic claim that needs judgement.
3. Bundles give provenance its own provenance, the hook for signatures.
4. Why, where and how provenance answer different questions about query outputs.
5. Semiring polynomials are universal: compute once, evaluate for counts, clearance, cost or deletion.
6. Validity constraints let machines detect impossible, and therefore suspicious, histories.

Next: putting theory into production pipelines.

04

SECTION FOUR

Lineage engineering

How modern data platforms capture, store, query and display lineage, with OpenLineage as the open standard.

Reference architecture and OpenLineage

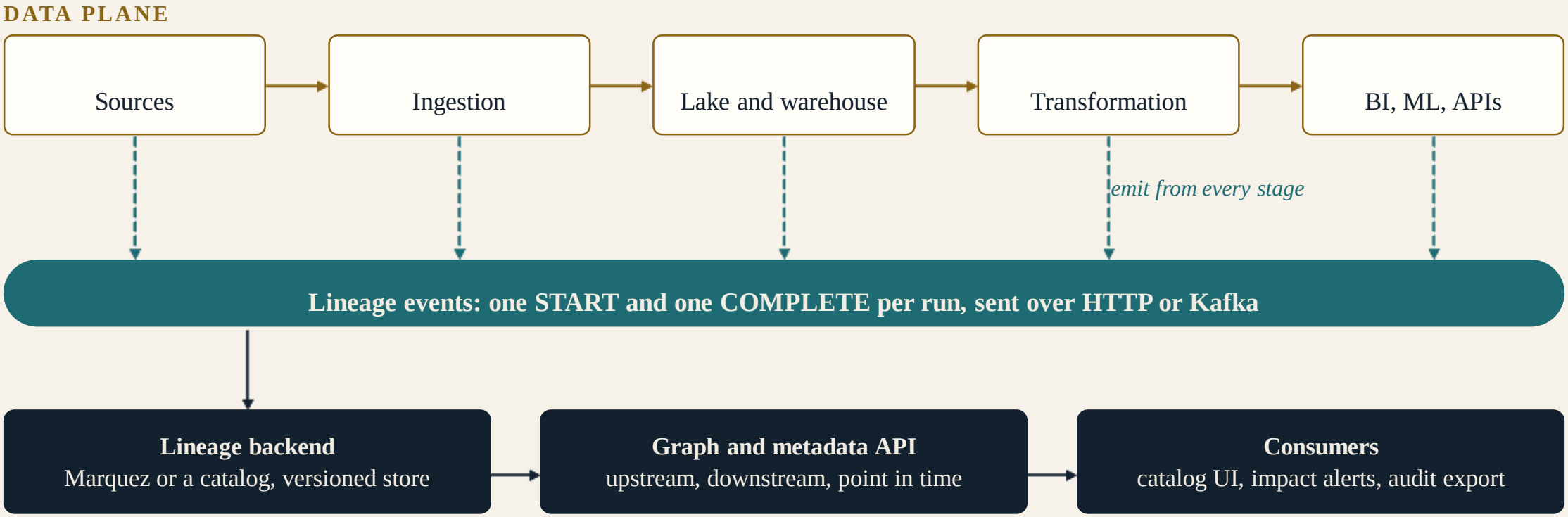
Column lineage and SQL parsing

dbt, Spark, Airflow and streaming

Graph storage, queries, UI and health



Reference architecture: a data plane and a lineage plane



The OpenLineage object model

Job

A recurring process, identified by **namespace** and **name**. Close to PROV's plan.

finance / revenue_rollup

Run

One execution of a job, identified by a unique **runId**. Close to PROV's activity.

run 0f3a…

Dataset

An input or output, identified by **namespace** and **name**. Close to PROV's entity.

s3://lake / marts/revenue

Facets

Typed, versioned JSON extensions attached to jobs, runs or datasets: schema, SQL, quality, ownership.

Run lifecycle carried by the *eventType* field



Inputs and outputs may be declared on any event; consumers merge them per run. A **parent** facet nests runs, for example a Spark job inside an Airflow task.

Anatomy of a run event

```
{
"eventType": "COMPLETE",
"eventTime": "[ISO 8601 timestamp]",
"producer": "https://example.org/etl/v2",
"run": { "runId": "[UUID]",
"facets": { "nominalTime": { … } } },
"job": { "namespace": "finance",
"name": "revenue_rollup" },
"inputs": [ { "namespace": "postgres://erp:5432",
"name": "public.orders" } ],
"outputs": [ { "namespace": "s3://lake",
"name": "marts/revenue",
"facets": { "schema": { … },
"columnLineage": { … } } } ]
}
```

State and time. When it happened and what stage.

Producer. Which integration emitted the event, and its version.

Run and job. This execution, and the recurring job it belongs to.

Inputs. Namespace encodes the system; name the asset inside it.

Outputs with facets. Schema and column lineage ride along.

Facets turn a lineage edge into a provenance record

Facet	Attached to	Provenance question answered
schema	Dataset	What fields and types existed?
dataSource	Dataset	Which system holds it?
columnLineage	Output dataset	Which input fields produced each field?
dataQualityMetrics	Input dataset	Row counts, nulls, distinct values
ownership	Job, dataset	Who is accountable?
sql	Job	What exact query ran?
sourceCodeLocation	Job	Which repository and commit?
parent	Run	Which orchestrating run spawned this?
errorMessage	Run	Why did it fail?
nominalTime	Run	Which business period was processed?

Custom facets

Add your own, for example a licence facet or a consent purpose facet, each with its own JSON schema URL so consumers can validate it.

Tip: the *dataQualityMetrics* row counts on inputs and outputs would have caught the lost Covid records in minutes.

An open standard with producers and consumers

PRODUCERS

- Apache Airflow provider
- Apache Spark listener
- dbt wrapper
- Apache Flink
- Dagster
- Great Expectations
- Custom Python and Java clients



An LF AI and Data Foundation project.
Instrument once, send anywhere.

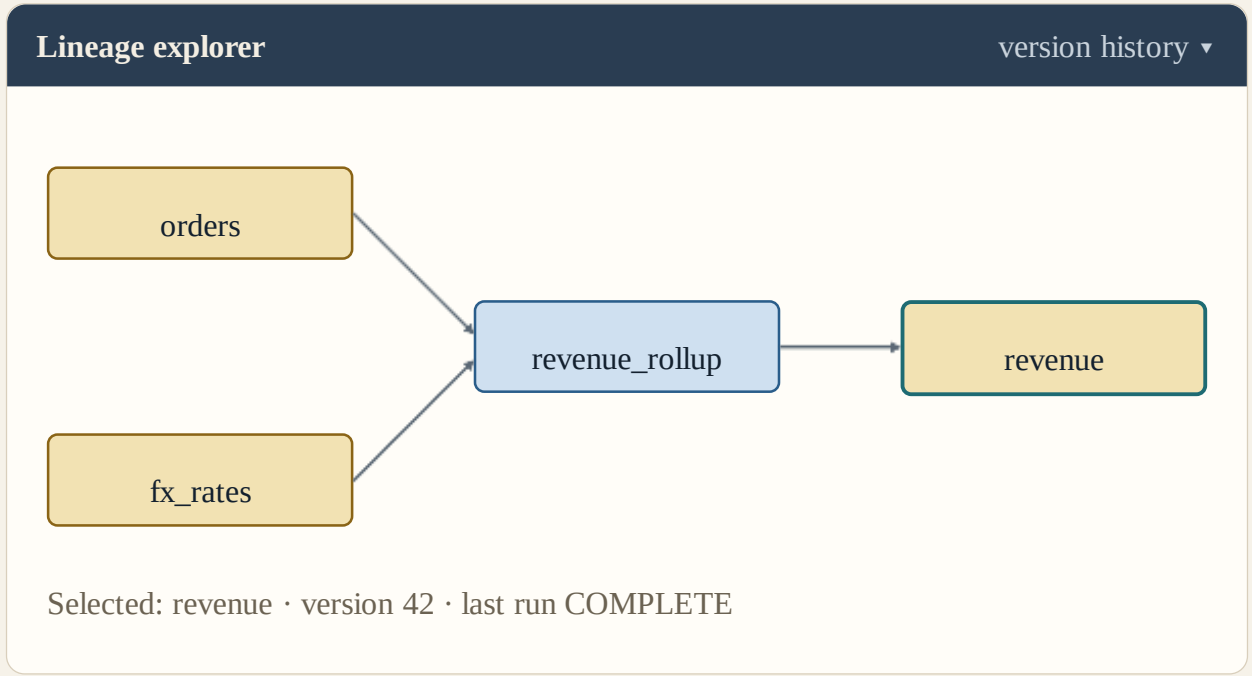
CONSUMERS

- Marquez, the reference backend
- Open source catalogs such as DataHub and OpenMetadata
- Cloud lineage services such as Google Cloud Dataplex and Amazon DataZone
- Commercial catalogs and observability tools

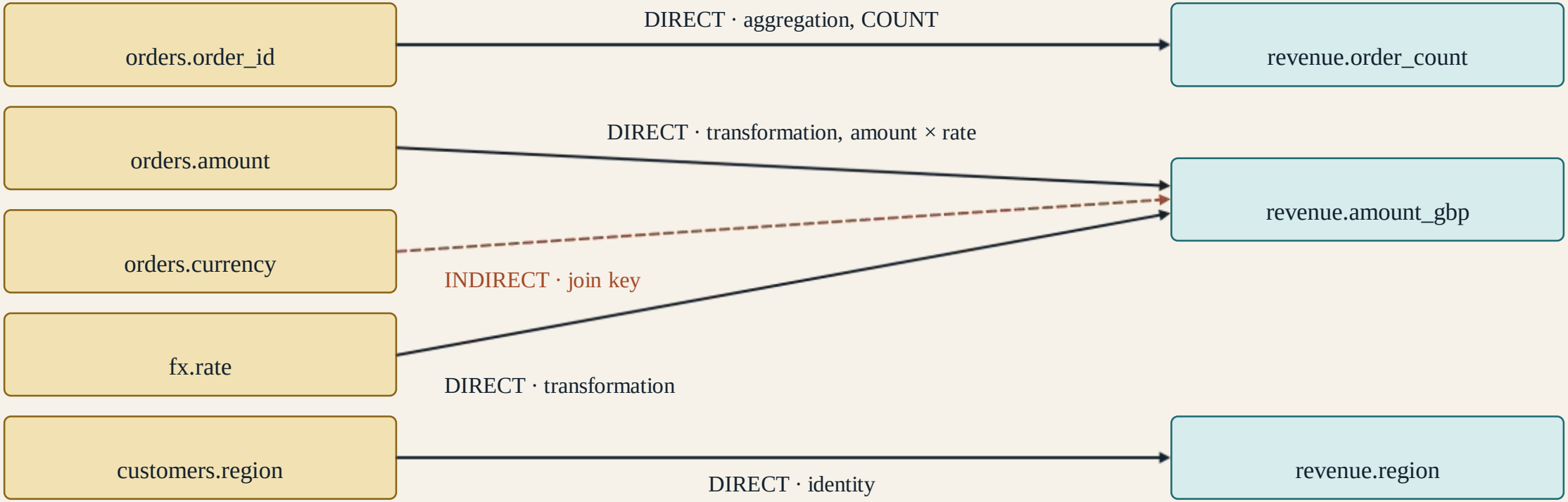
Marquez: versioned lineage with time travel

- **Run level metadata.** Every run is stored with its inputs, outputs and facets.
- **Job and dataset versions.** Changes to code or schema create new versions automatically.
- **Time travel.** View the lineage graph as it was for any earlier version.
- **Lineage API and UI.** HTTP API for graph queries; a web UI to explore it.

Quick start runs in Docker with a PostgreSQL store. Version 0.49 is aligned with OpenLineage specification 2.0.2.



Column lineage: direct and indirect influence



The OpenLineage column lineage facet records both kinds: **direct** (identity, transformation, aggregation) where values flow, and **indirect** (join, filter, group by, sort, window, conditional) where columns shape *which* rows appear.

How SQL parsing derives column lineage

1 — Parse
Dialect aware parser builds an abstract syntax tree.

→

2 — Resolve
Look up schemas to expand aliases and stars.

→

3 — Trace
Walk expressions from each output column to its sources.

→

4 — Classify
Label each edge direct or indirect, and its subtype.

→

5 — Emit
Write a column lineage facet or catalog edges.

Common open source parsers

sqlglot: Python, many dialects, lineage helpers.
sqllineage: Python, table and column lineage.
OpenLineage SQL parser: Rust core used by several integrations.

Where parsing breaks

SELECT star without a schema; dynamic SQL built at runtime; stored procedures and UDFs; temporary tables; vendor specific syntax; logic hidden in Python around the SQL.

Four ways to collect lineage, compared

Dimension	Static parsing	Runtime instrumentation	Query log mining	Manual curation
Source	Code and SQL text	Listeners in engines	Warehouse query history	People and documents
Reflects	What is written	What actually ran	What actually ran	What people believe
Run versions	No	Yes	Partly	No
Coverage	SQL only	Instrumented tools	One warehouse	Whatever is maintained
Effort	Low	Medium	Low	High, ongoing
Main risk	Misses dynamic logic	Gaps in uninstrumented tools	Blind outside warehouse	Rapid decay
Best for	CI checks before deploy	Audit grade evidence	Fast bootstrap	Spreadsheets, vendors, people

Recommended blend: runtime instrumentation as the source of truth, static parsing in CI to predict impact before merge, log mining to find what you missed, and manual curation only for the truly offline edges.

dbt: lineage declared by construction

models/marts/revenue.sql

```
select o.order_date,
sum(o.amount * fx.rate) as amount_gbp
from {{ ref('stg_orders') }} o
join {{ ref('stg_fx') }} fx
on o.currency = fx.currency
group by 1
```



ref and source make dependencies explicit, so the DAG is exact at model level.

Artefacts: the manifest and run results JSON files capture the graph and each run’s outcome.

Exposures declare downstream dashboards and models, closing the loop to consumers.

OpenLineage: a dbt wrapper emits run events, with column lineage parsed from compiled SQL.

Spark: reading lineage from the logical plan

1. Your job builds DataFrames; Spark compiles them into a **logical plan**.
2. The OpenLineage **listener** receives query execution events.
3. Visitors walk plan nodes to find read and write relations and column dependencies.
4. Events are emitted with dataset facets, including table format versions where available.

SPARK CONFIGURATION

```
spark.extraListeners =
io.openlineage.spark.agent.OpenLineageSparkListener
spark.openlineage.transport.type = http
spark.openlineage.transport.url = [backend URL]
spark.openlineage.namespace = finance
```

Blind spots

Low level RDD operations, opaque UDFs, data collected to the driver and written by plain Python, and external side effects.

Orchestrators: the natural home of run provenance

Built in OpenLineage provider

Airflow ships an OpenLineage provider package. Configure a transport and every task run emits START and COMPLETE or FAIL events.

Operator level extraction

SQL, transfer and cloud operators report their own inputs and outputs through OpenLineage methods on start and completion.

Parent and child runs

A Spark or dbt job launched by a task links back through the parent facet, giving one graph across tools.

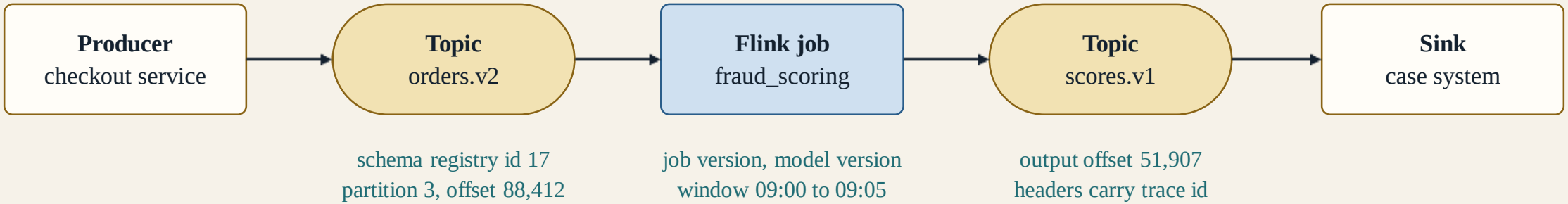
Assets in Airflow 3

What were called datasets are now assets. Declaring them drives data aware scheduling and doubles as declared lineage.

The orchestrator knows who triggered a run, which DAG version ran and which logical date was processed. Capture all three; they are the agent, plan and time of PROV.

Streaming: provenance for data that never stops

The coordinates (topic, partition, offset) identify every event exactly, forever



Long running runs

A streaming job is one run lasting months. Emit periodic RUNNING events and new versions when code or config change.

Record level tracing

Propagate a trace or correlation id in message headers so one decision can be followed end to end.

Retention trap

Topics expire. If provenance points to offsets that no longer exist, archive the events or the evidence is gone.

Where to store the lineage graph

Option	Strengths	Weaknesses	Fits when
Relational with recursive queries	Familiar, transactional, easy to version rows	Deep traversals get slow	Up to millions of edges; Marquez uses PostgreSQL
Property graph database	Fast variable depth traversals, expressive queries	Another system to operate	Large, highly connected estates
RDF triple store	Native PROV O, standard SPARQL, reasoning	Steeper skills curve	Knowledge graph strategy, research
Search index plus cache	Instant search and neighbourhood views	Not a source of truth	Catalog UIs on top of any store
Append only event log	Immutable, replayable, audit friendly	Needs materialised views to query	Pair with any of the above

Pattern: keep the raw lineage events in an immutable log as the evidence of record, and build the queryable graph as a derived view that can always be rebuilt.

Three queries every lineage store must answer

1 Upstream, in SQL

```
WITH RECURSIVE up(ds, depth) AS (
SELECT input_ds, 1 FROM edges
WHERE output_ds = 'mart_revenue'
UNION
SELECT e.input_ds, u.depth + 1
FROM edges e JOIN up u
ON e.output_ds = u.ds
WHERE u.depth < 10
)
SELECT DISTINCT ds FROM up;
```

2 Downstream, in Cypher

```
MATCH (s:Dataset {name: 'erp.orders'})
[:FEEDS*1..6]>(d)
RETURN DISTINCT d.name, d.owner
```

Relationship arrow shown simplified.

3 Point in time

```
SELECT input_ds, input_version
FROM run_inputs
WHERE run_id = 912;
```

Always bound traversal depth, return owners alongside assets, and answer point in time questions from **run records**, not from today’s graph.

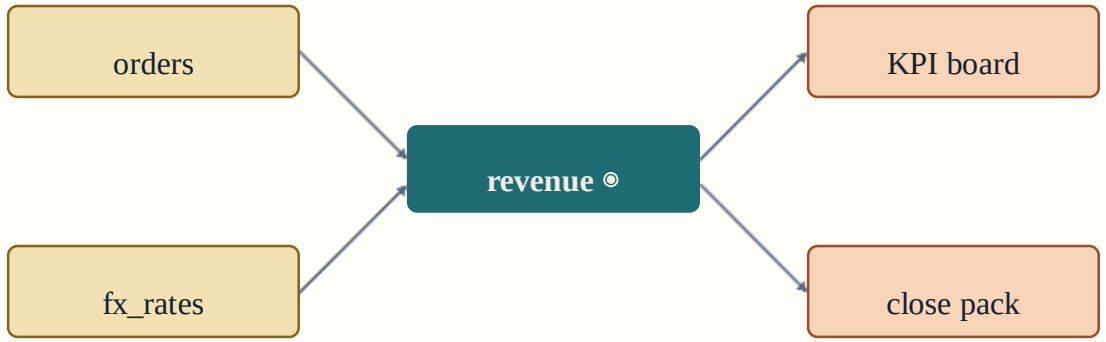
Designing a lineage view people actually use

Search assets

revenue

Filters

☑ Hide hub tables
☑ Owners only: Finance
☐ Show column level
☐ Show failed runs



Time slider: lineage as of run 912, 6 Nov 2026

revenue

Owner: Finance Data
Last run: COMPLETE, 09:00
Freshness: 2 hours
Quality: 14 of 14 checks pass
Licence: internal
Contains personal data: no
Version: 42

A lineage health scorecard you can adopt today

Metric	Definition	Target	Current
Coverage	Share of production jobs emitting lineage events	[__ %]	[__ %]
Column depth	Share of critical outputs with column lineage	[__ %]	[__ %]
Orphans	Datasets with no known producer	[__]	[__]
Freshness	Median delay from run end to lineage visible	[__ min]	[__ min]
Agreement	Edges confirmed by both static and runtime methods	[__ %]	[__ %]
Ownership	Lineage nodes with a named accountable owner	[__ %]	[__ %]
Name collisions	Same physical asset appearing as several nodes	[__]	[__]

Placeholders are deliberate: set targets by risk tier. **Orphans and name collisions** are usually the quickest wins and the clearest signs of broken capture.

Six anti patterns, and the fix for each

Lineage as a drawing

A diagram maintained by hand in a slide tool.

Fix: generate from runtime events.

No versions

Edges between names, never between versions.

Fix: record run inputs with versions.

The hairball

Every node shown at once; nobody can read it.

Fix: focus, filter and bound depth.

Ownerless nodes

You find the impact but not whom to tell.

Fix: ownership facet mandatory.

Blind trust in parsing

Parsed edges treated as proven fact.

Fix: label edge confidence and method.

Warehouse only

Lineage stops at the warehouse walls.

Fix: instrument ingestion, BI and ML.

The common thread: lineage that is not **automatic, versioned, owned and honest about its confidence** will not survive its first audit.

SECTION 04 RECAP

Engineering lineage that holds up

1. Separate the data plane from the lineage plane, joined by an open event format.
2. OpenLineage gives jobs, runs, datasets and facets; facets make it provenance.
3. Column lineage needs both direct and indirect edges.
4. Blend runtime, static and log based collection; disagreement is a signal.
5. Store events immutably; build graphs as rebuildable views.
6. Measure lineage health like any other production system.

Next: capture techniques beyond the orchestrator.

05

SECTION FIVE

Capture techniques

The full spectrum of ways to record history: from annotations and decorators to operating system tracing, versioned tables and content addressing.



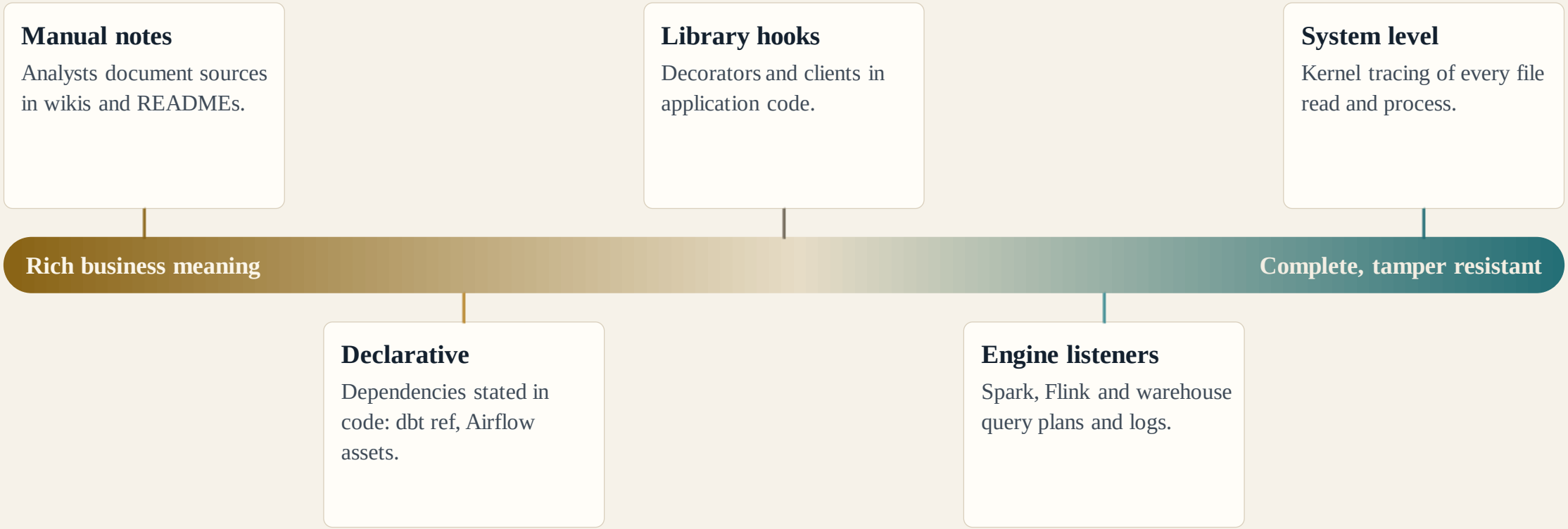
Eager versus lazy, system level capture

Instrumentation patterns and CDC

Bitemporal data, versioning, snapshots

Identifiers, overhead and privacy

The capture spectrum: meaning versus completeness



Left of the spectrum, records explain *why* but are incomplete and easily wrong. Right of it, records are complete but say little about purpose. **Combine at least one technique from each half.**

Eager or lazy: when is provenance computed?

Eager

Annotations travel with data as it is processed. Every output carries its provenance at birth.

row + tag → row + tags

Gains: instant answers, survives source deletion.

Costs: storage and runtime overhead on every row.

Lazy

Store only the query, inputs and versions. Reconstruct fine grained provenance on demand.

question → replay

Gains: near zero runtime cost.

Costs: needs immutable inputs and deterministic code.

Practical hybrid: capture coarse provenance eagerly for every run, and keep inputs versioned so that row and cell level provenance can be rebuilt lazily when an investigation needs it.

Whole system provenance: watching the kernel

PASS
Harvard’s provenance aware storage system recorded file and process ancestry inside the file system.

SPADE
Collects provenance from audit logs across hosts and exports graphs, including PROV.

CamFlow
Linux security module capturing information flow with low overhead.

eBPF tooling
Modern kernels let tracers observe system calls safely in production.

Where it shines
Security forensics and intrusion detection: attack graphs show how a malicious file spread. Nothing escapes capture because applications need no cooperation.

Where it struggles
Graphs grow enormous, inside databases the kernel sees only page reads, and business meaning is lost. This is the **semantic gap**.

The semantic gap: same event, four levels of meaning

Kernel	process 4412 opened /data/c.parquet, read 18 MB, wrote /tmp/x91
Engine	Spark stage 7 scanned table customers, joined events, wrote features
Pipeline	job churn_features run 57 for logical date 1 November used customers v12
Business	Ada refreshed retention features for the Q4 campaign, under the marketing consent purpose

Lower levels are **automatic and complete**, but cannot state purpose or consent.

Upper levels answer regulators, but only if someone **links them downward** with shared identifiers.

Five instrumentation patterns

Pattern	How it works	Example	Watch out for
Decorator	Wrap a function; record inputs, outputs and code version on each call	Python decorator on a training step	Developers forgetting to apply it
Middleware	Intercept requests in a service framework	API gateway logging data access	Missing payload versions
Listener	Subscribe to engine lifecycle callbacks	Spark or Flink listeners	Engine upgrades changing APIs
Sidecar	A companion process observes traffic or files	Container sidecar in Kubernetes	Encrypted traffic is opaque
Change capture	Read the database's own write log	Debezium on a PostgreSQL WAL	Log retention and schema drift

Golden rule: provenance capture must never break the pipeline. Emit asynchronously, buffer locally and alert on loss.

But in regulated flows, consider failing closed: no provenance, no publication.

A provenance decorator in Python

```
import functools, hashlib, json, subprocess, time, uuid

def provenance(emit):
def wrap(fn):
@functools.wraps(fn)
def inner(*paths, **params):
run = str(uuid.uuid4())
ins = {p: digest(p) for p in paths}
commit = subprocess.check_output(
["git", "rev", "parse", "HEAD"]).strip()
t0 = time.time()
out = fn(*paths, **params)
emit({"run": run, "fn": fn.__name__,
"inputs": ins, "output": digest(out),
"params": params, "code": commit,
"seconds": time.time() t0})
return out
return inner
return wrap
```

Run id joins this record to logs and lineage events.

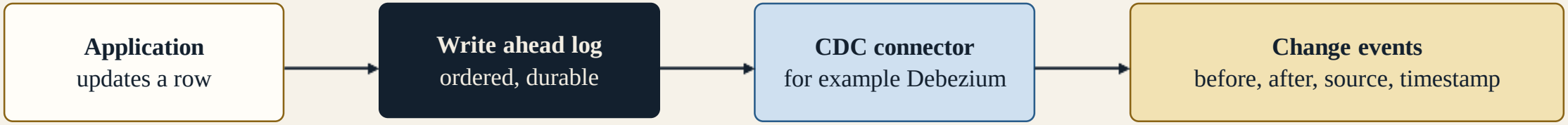
Content digests pin exact inputs and output, not just names.

Git commit pins the code; params pin the configuration.

emit is pluggable: OpenLineage, a log, or a signed store.

Simplified for the slide: the git subcommand is normally written with a hyphen, and the duration as a subtraction. Add error handling and a dirty tree check.

Change data capture: the database testifies



ONE CHANGE EVENT, SIMPLIFIED

```
{ "op": "u",
"before": { "id": 7, "tier": "silver" },
"after": { "id": 7, "tier": "gold" },
"source": { "db": "crm", "table": "customers",
"lsn": 7340032, "txId": 5521 } }
```

Complete: every committed change, including those made by hand.

Ordered: the log position orders events exactly.

Missing: who and why. Add the user and reason in application context columns.

Bitemporal data: what was true, and when we knew it

Row	Salary	Valid from	Valid to	Recorded from	Recorded to
1	40,000	1 Jan	open	2 Jan	15 Mar
2	40,000	1 Jan	1 Mar	15 Mar	open
3	45,000	1 Mar	open	15 Mar	open

Valid time

When the fact was true in the world. The raise took effect on 1 March.

Transaction time

When the database learned it. HR only entered it on 15 March.

The audit question

“What did our 10 March payroll report believe?” Only transaction time can answer it.

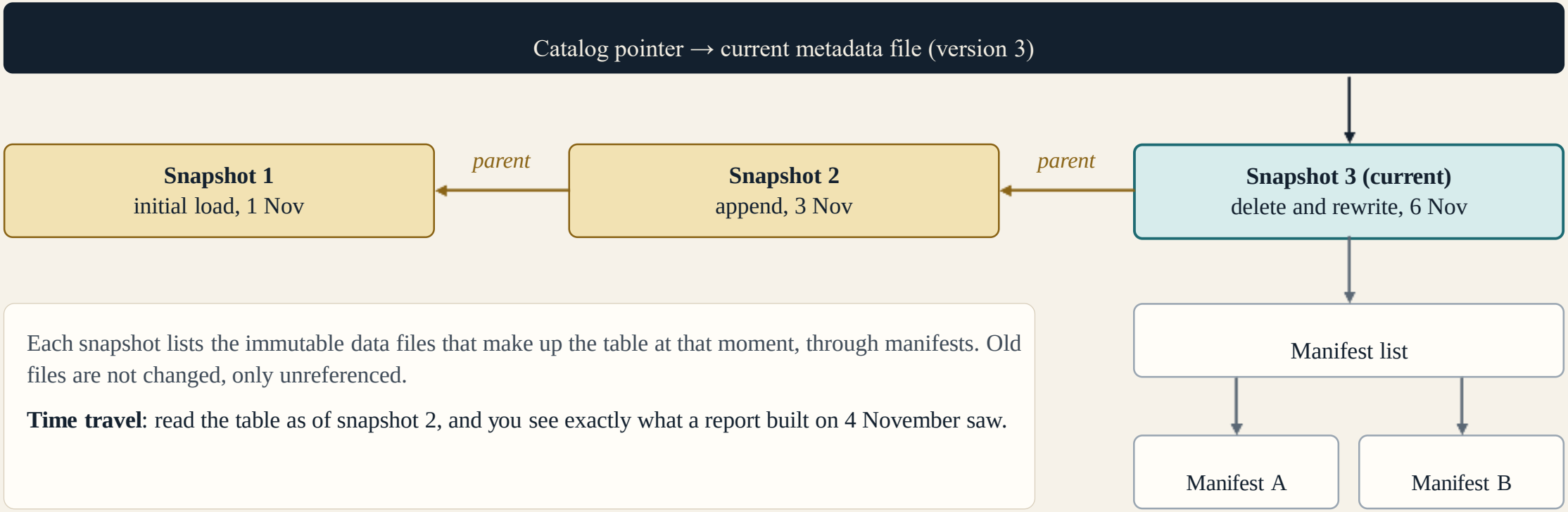
SQL:2011 standardised **system versioned** tables for transaction time and **application time periods** for valid time; several databases implement one or both.

Data versioning tools: making history addressable

Tool	Unit of version	Provenance feature	Typical use
Apache Iceberg	Table snapshot	Snapshot log, time travel queries, branches and tags	Lakehouse analytics tables
Delta Lake	Table version	Transaction log with operation history and time travel	Lakehouse on Spark platforms
Apache Hudi	Commit on a timeline	Incremental queries between instants	Upsert heavy pipelines
lakeFS	Repository commit	Git style branches and merges over object storage	Isolated pipeline testing
Project Nessie	Catalog commit	Versioning across many tables at once	Consistent multi table states
DVC	File or directory hash	Data tracked beside code in git	ML datasets and models

The provenance link: record the snapshot or commit ID in every lineage event. A table name tells you *what*; a snapshot ID tells you *exactly which*.

How a snapshot chain preserves every past state



Each snapshot lists the immutable data files that make up the table at that moment, through manifests. Old files are not changed, only unreferenced.

Time travel: read the table as of snapshot 2, and you see exactly what a report built on 4 November saw.

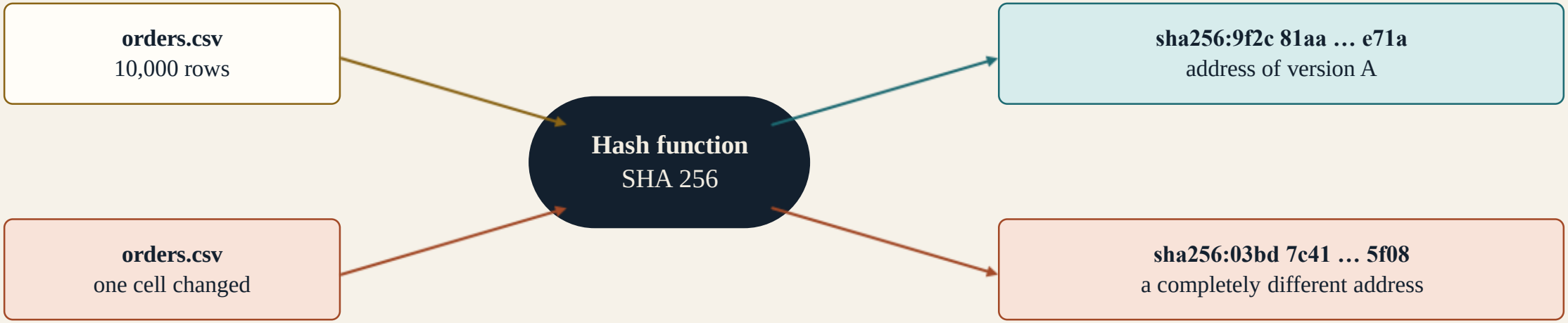
Identifiers: provenance is only as good as its names

Identifier type	Example	Strength	Weakness
Human name	sales_final_v2	Readable	Ambiguous, reused, renamed
Random UUID	a 128 bit random value	Unique without coordination	Says nothing about content
Namespaced URI	s3://lake/marts/revenue	Locates the asset	Location can change or be reused
Content hash	sha256:9f2c…	Proves exact bytes; immutable	Changes with every edit
Persistent ID	DOI, ARK, Handle	Resolvable for decades	Needs an organisation behind it

Use two: a stable logical name for the asset, plus a content hash or snapshot ID for each version.

Never reuse a name for different content. Reuse silently corrupts every past provenance record.

Content addressing: the name is the fingerprint



Immutable by design. Content cannot change without its name changing.

Self verifying. Anyone can recompute the hash to confirm they hold the right bytes.

Everywhere already: git objects, container image digests, IPFS, DVC caches.

The minimum viable provenance record, by tier

Field	Tier 3: exploratory	Tier 2: business	Tier 1: regulated
Asset name and owner	Required	Required	Required
Producing job and run id	Recommended	Required	Required
Input names and versions	Names only	Required	Required
Code commit and parameters	Optional	Required	Required
Column lineage	Optional	Recommended	Required
Content hash of outputs	Optional	Recommended	Required
Purpose, licence, legal basis	If personal data	Required	Required
Quality checks and results	Optional	Required	Required
Signature over the record	No	Optional	Required
Retention of inputs for replay	No	Policy based	Audit horizon

Use this as a starting template; adjust tiers to your regulatory exposure and risk appetite.

Taming provenance overhead: six levers

Tier by risk

Rich capture only where the stakes justify it.

Deduplicate

Store repeated job definitions and schemas once; reference by hash.

Summarise

Collapse thousands of identical runs into one edge with counts.

Tiered retention

Hot for weeks, warm for months, cold archive for the audit horizon.

Go lazy

Record enough to replay rather than every fine detail.

Sample

Record level provenance for a sample of streaming events, all of them for flagged ones.

Measure first. Track provenance storage per pipeline and event emission latency, so cost discussions rest on numbers rather than fear.

Provenance is itself sensitive data

It reveals people
Who ran which query, when, on whose records. That is workplace monitoring data.

It leaks content
Captured SQL can contain literal values: names, account numbers, diagnoses.

It exposes structure
A full lineage graph is a map of your crown jewels for an attacker.

Control access
Provenance inherits the classification of the data it describes.

Redact and abstract
Parameterise literals out of captured SQL; show abstracted views to wider audiences.

Govern its retention
Set lawful retention and inform staff, as for any personal data.

SECTION 05 RECAP

Capturing history without drowning in it

1. Combine semantic and complete capture; neither alone is sufficient.
2. Shared identifiers bridge the semantic gap between levels.
3. Bitemporal tables and snapshots make the past addressable.
4. Content hashes turn identifiers into verifiable evidence.
5. Tier capture by risk and use lazy replay to control cost.
6. Treat provenance as sensitive data with its own controls.

Next: how to make provenance trustworthy against tampering.

06

SECTION SIX

Integrity and trust

Provenance can be forged, altered or silently omitted. Cryptography turns a record you must believe into evidence you can check.

Threat model for provenance

Hashes, Merkle trees and signatures

Transparency logs and attestations

Blockchain, hardware and trust anchors



Six ways provenance can lie

Threat	Example	Primary countermeasure
Forgery	A record claims an origin that never happened	Digital signatures by the producing system
Tampering	A field is edited after the fact	Hashes and signatures over the whole record
Omission	An inconvenient step is never recorded	Hash chains, transparency logs, capture below the application
Replay	An old valid record is presented for new data	Bind records to content digests and timestamps
Repudiation	A party denies having performed a step	Signatures with individually held keys
Collusion	Several insiders rewrite history together	Independent witnesses and public logs

Hardest of all: omission. Cryptography proves that what was recorded is unaltered; it cannot prove that everything was recorded.

Cryptographic hashes: three properties that matter

PREIMAGE RESISTANCE

One way

Given a digest, you cannot find an input that produces it.

SECOND PREIMAGE

No substitutes

Given a file, you cannot craft a different file with the same digest.

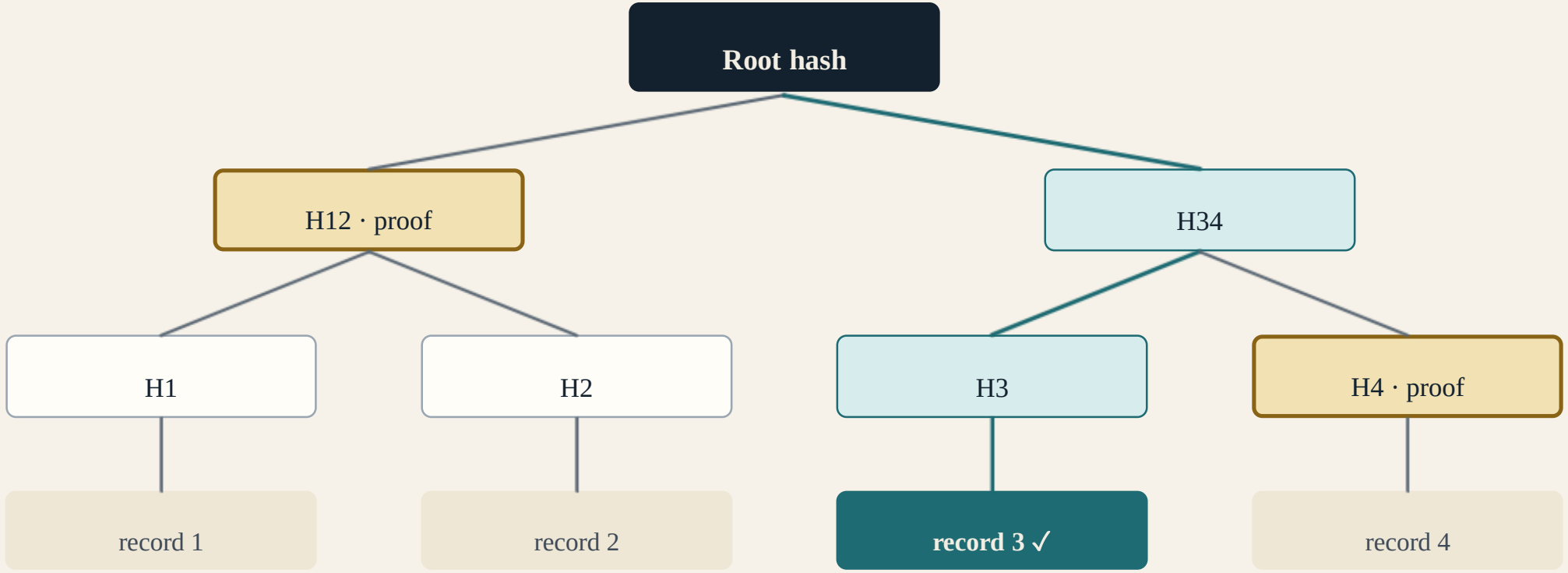
COLLISION RESISTANCE

No twins

You cannot find any two inputs that share a digest.

Algorithm	Digest size	Status for provenance use
MD5	128 bits	Broken for collisions; use only as a non security checksum
SHA 1	160 bits	Practical collisions shown in 2017; avoid for new designs
SHA 256	256 bits	Current default in git object formats, containers and signing
SHA 3 and BLAKE3	256 bits and more	Strong modern alternatives; BLAKE3 is very fast

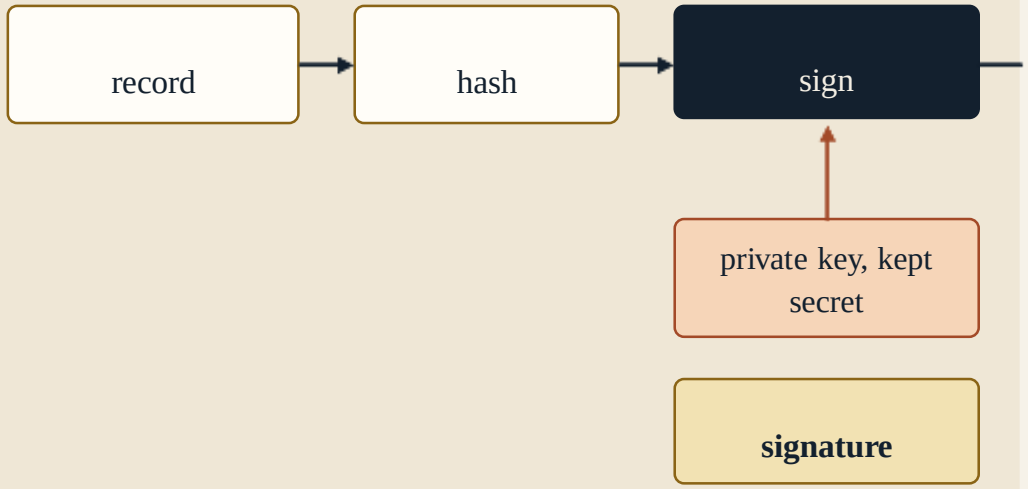
Merkle trees: one root commits to everything



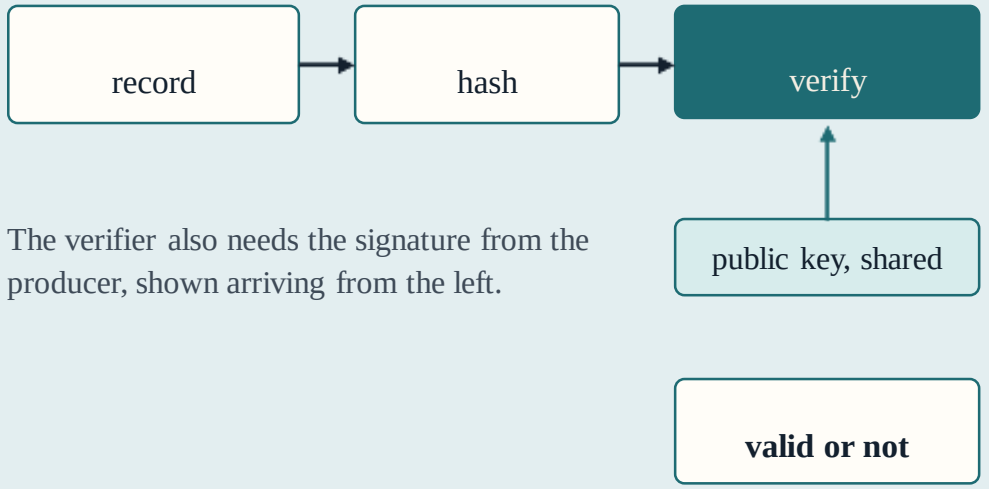
Inclusion proof: to show record 3 is in the set, hand over only H4 and H12. The verifier recomputes H3, then H34, then the root, and compares. Proof size grows with the logarithm of the number of records.

Digital signatures bind a record to its author

PRODUCER SIGNS



ANYONE VERIFIES



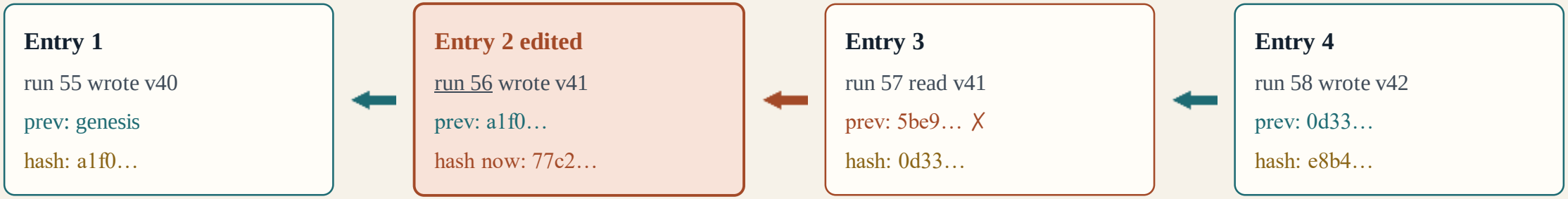
The verifier also needs the signature from the producer, shown arriving from the left.

Algorithms: Ed25519 and ECDSA are common; RSA remains widespread.

Timestamps: an RFC 3161 authority proves the signature existed at a time.

The hard part: key custody, rotation and revocation, not the mathematics.

Hash chained logs make deletion and reordering visible



Each entry stores the hash of the one before. Editing entry 2 changes its hash to 77c2…, but entry 3 still points to the original 5be9…: **the break is detectable by anyone who walks the chain**.

Limitation: whoever controls the whole log can rewrite every entry after the edit. Rehashing a chain is cheap.

Remedy: regularly publish or countersign the latest hash somewhere the operator cannot change.

Transparency logs: public, append only, verifiable



Inclusion proof
This entry is in the log under this tree head.

Consistency proof
The new tree extends the old one; nothing was removed.

In production
Certificate Transparency, RFC 6962; Sigstore's Rekor for software signatures.

Do you need a blockchain for provenance?

Q1
Do several parties write records?
No → a signed database

Q2
Is there no operator all parties trust?
No → a transparency log

Q3
Can data be shared without privacy harm?
No → store hashes only

Then
A shared ledger may genuinely help.

The oracle problem

A ledger guarantees that a record was not changed after entry. It cannot guarantee the record was true when entered. A mislabelled crate stays mislabelled, immutably.

The pragmatic pattern

Keep provenance in your own systems, and periodically anchor a Merkle root in a public timestamping service or chain. Cheap, private, independently checkable.

Evident, resistant, proof: say exactly what you mean

Tamper evident

Changes can happen but are always detectable.

Examples: hash chains, signatures, Merkle logs, a sealed evidence bag.

Achievable and the right goal for most provenance.

Tamper resistant

Changes are hard and costly to make.

Examples: write once storage, object lock, hardware security modules, separation of duties.

Complements evidence with prevention.

Tamper proof

Changes are impossible.

Examples: none in practice. Any system with an administrator can be subverted.

A marketing word. Challenge it in procurement.

Write requirements as **tamper evident and tamper resistant**, and state against which actor: an external attacker, an operator, or a colluding group.

Attestations: signed, typed claims about artefacts

ENVELOPE · signed

STATEMENT

subject: churn_model.onnx, sha256:4c1e…

predicateType: a URI naming the kind of claim

predicate: the claim itself, for example how the model was built and from which datasets

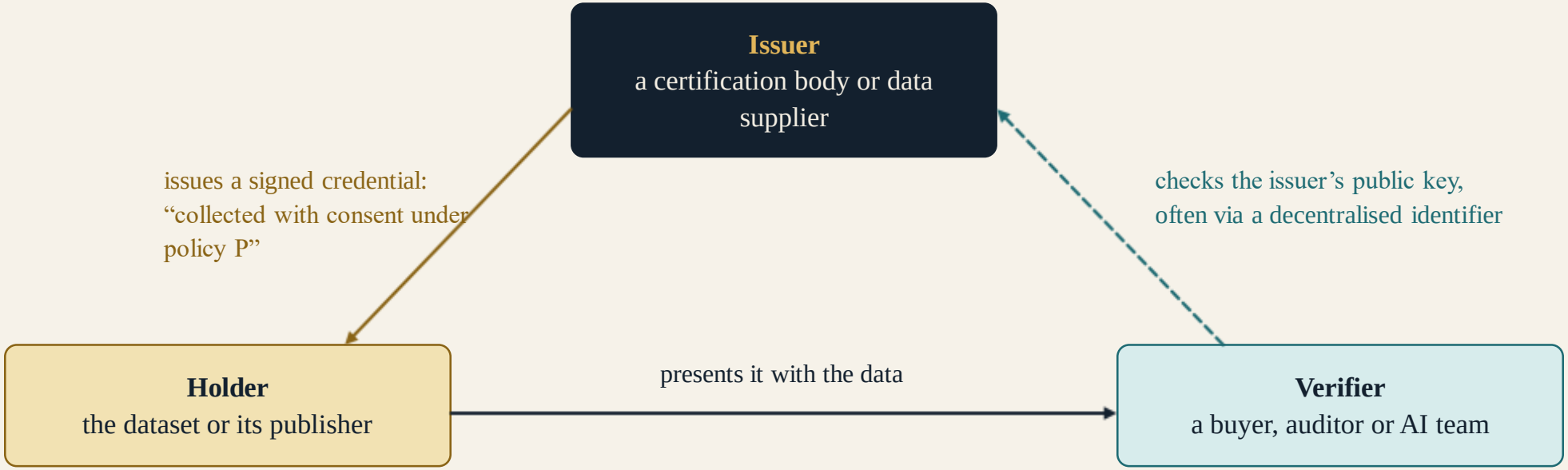
signatures: one or more, over the whole payload

Common predicate types

- SLSA build provenance
- Software bill of materials
- Test and evaluation results
- Vulnerability scan results
- Your own: dataset licence review

This layered structure comes from the **in toto attestation framework**. Because the subject is a digest, the claim attaches to exact bytes, not to a name that might be reused.

Verifiable credentials: portable claims about sources



The W3C **Verifiable Credentials Data Model** standardises the format; selective disclosure lets a holder prove one claim without revealing the rest. Useful when provenance must travel across organisations.

Moving trust into hardware, closer to the source

Hardware security modules

Signing keys never leave tamper resistant hardware, so stolen servers cannot forge provenance.

Trusted execution

Confidential computing enclaves produce remote attestations proving which code ran on which data.

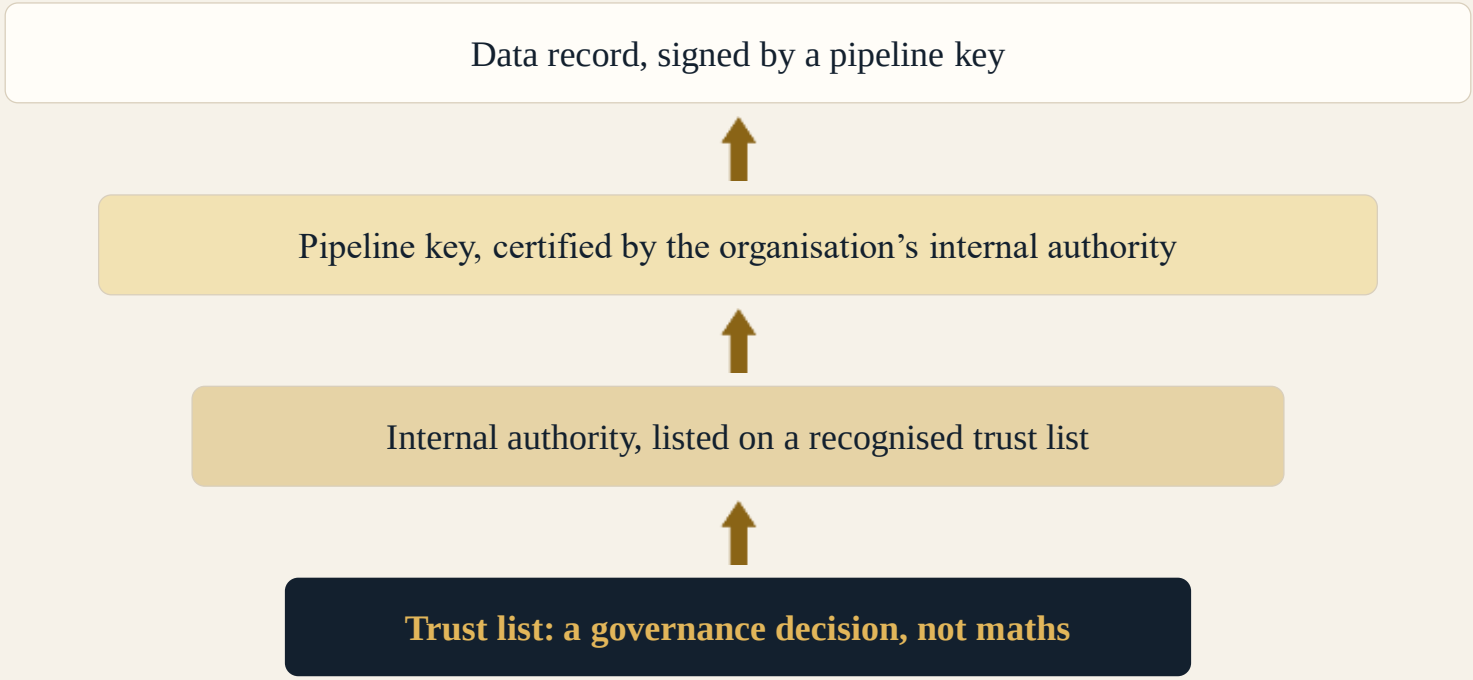
Signing at capture

Several camera and phone makers now sign images in the device, the first link of a content credential.

Why it matters: software provenance can be forged by whoever controls the software. Hardware roots make forgery require physical compromise.

But: a signed photo of a staged scene is still a staged scene. Hardware proves capture, not truth.

Who signs the signers? The trust anchor problem



Every chain ends at an anchor you accept on judgement. In C2PA, for example, the coalition operates trust lists of recognised signers, and its interim trust list was frozen to new entries from 1 January 2026 as a formal list took over.

SECTION 06 RECAP

From claims to evidence

1. Start from a threat model: which actor, which lie.
2. Hashes pin content; signatures pin authorship.
3. Merkle logs with witnesses defeat silent rewriting.
4. Attestations package signed claims about exact bytes.
5. Blockchains rarely beat anchoring; none solves the oracle problem.
6. Every chain ends in a governed trust anchor.

Next: provenance for machine learning and AI.

07

SECTION SEVEN

Machine learning and AI

Models are compiled from data. Their provenance decides whether they are lawful, safe, fair and reproducible.

Datasheets, model cards and standards

Training data audits and licensing

LLM pipelines and RAG provenance

Synthetic data, unlearning, AI BOMs



Eight provenance checkpoints in the ML lifecycle

1

Sourcing

Origin, licence, consent, collection date and method.

2

Labelling

Annotators, guidelines version, agreement, adjudication.

3

Preparation

Filters, deduplication, PII removal, balancing choices.

4

Splits

Exact train, validation and test membership with seeds.

5

Training

Code commit, hyperparameters, environment, hardware, base model.

6

Evaluation

Benchmarks and versions, subgroup metrics, red team results.

7

Release

Approver, model card, signed artefact digest.

8

Inference

Model version per decision, inputs, retrieved sources, overrides.

Checkpoints 1 to 4 are **data provenance**; 5 to 7 are **model provenance**; 8 is **decision provenance**, the record that lets you explain an individual outcome later.

Datasheets for Datasets: seven chapters of a dataset’s life

Motivation
Why was it created, by whom, funded by whom?

Composition
What do instances represent? Missing data? Sensitive content?

Collection
How, when and from whom? Was consent obtained?

Preprocessing
Cleaning and labelling done; is the raw data kept?

Uses
Intended tasks; uses to avoid.

Distribution
Licence, export controls, how it is shared.

Maintenance
Who maintains it, how errata and erasures are handled.

Origin
Gebru and colleagues, 2018; published in Communications of the ACM, 2021.

Inspired by electronics: every component ships with a datasheet of operating characteristics. Datasets should too.

Its real value is the questions it forces creators to answer, many of which are provenance questions in disguise.

Model cards: the model’s provenance and limits in one page

Model card · churn_model v3

Model details · owner, version, type, date

Intended use · and out of scope uses

Factors · groups, instruments, environments

Metrics · and why they were chosen

Evaluation data · with provenance

Training data · with provenance

Quantitative analyses · disaggregated results

Ethical considerations

Caveats and recommendations

Proposed by Mitchell and colleagues at the FAT* conference in 2019, model cards are now standard on model hubs.

Link, don’t paste. Training and evaluation data sections should point to dataset versions and their datasheets by identifier.

Generate, don’t type. Populate metrics and data references from experiment tracking so the card cannot drift from the truth.

Common gap: “trained on public web data” is not provenance.

The documentation family for AI data

Framework	Origin	Form	Distinctive strength
Datasheets	Gebru et al., 2018	Questionnaire	Breadth of creator questions
Data Statements	Bender and Friedman, 2018	Narrative	Language variety and speaker demographics for NLP
Dataset Nutrition Label	MIT and Harvard, 2018	Label	At a glance summary for consumers
AI FactSheets	IBM, 2019	Supplier declaration	Lifecycle facts for the whole AI service
Data Cards	Google, 2022	Structured cards	Audience layered, explains rationale
Croissant	MLCommons, 2024	JSON LD metadata	Machine readable; loads straight into frameworks
Hub cards	Model and dataset hubs	Markdown with YAML	Ubiquity at the point of download

Trend: from documents humans read to metadata machines validate. Croissant and standardised provenance fields make documentation enforceable in pipelines.

07 · MACHINE LEARNING AND AI · INDUSTRY STANDARD

The Data and Trust Alliance Data Provenance Standards

22

metadata fields in version 1.0.0, grouped into three standards

50+

organisations tested and validated the standards before release in July 2024

The eight original categories, proposed November 2023

Source	Legal rights
Privacy and protection	Generation date
Data type	Generation method
Intended use and restrictions	Lineage, with a unique ID

Developed by 19 member organisations including American Express, IBM, Mastercard, Pfizer, UPS and Walmart. IBM tested them in clearing datasets for foundation model training.

From alliance to open standard: the OASIS DPS committee

Nov 2023
D&TA proposes eight provenance standards

→

Jul 2024
Version 1.0.0 released after broad testing

→

2024 to 2025
Mapped with EDM Council and AI Alliance; fields reach Hugging Face cards

→

Apr 2025
OASIS Technical Committee convenes

Sponsors

Cisco, the Data and Trust Alliance, IBM, Intel, Microsoft and Red Hat, among others.

Planned deliverables

Committee specifications for standard provenance tags, plus implementation guides covering provenance, pedigree and lineage.

Notable scope

Geolocation of origin, from coordinates to political boundaries, to support trust judgements by place.

Licence laundering: how restrictions vanish downstream



Evidence

The Data Provenance Initiative’s 2023 audit traced more than 1,800 fine tuning datasets and found licences on popular hosting sites frequently omitted or miscategorised, often more permissive than the original authors intended. Its follow up study in 2024 documented a rapid rise in web publishers restricting AI crawlers.

Controls

Trace every dataset to its original source; record licence as an inherited, most restrictive property; review aggregations; snapshot robots and terms status at crawl time.

Provenance through a language model data pipeline

1 Acquire
URL, fetch time, robots and terms status, licence, content hash

2 Filter
Quality and safety classifier versions, thresholds, drop reasons

3 Deduplicate
Near duplicate clusters, kept representative, method parameters

4 Protect
PII detector version, redaction counts, opt out lists applied

5 Mix
Source weights, proportions, synthetic share, contamination checks

6 Tokenise
Tokenizer version, shard manifests with digests

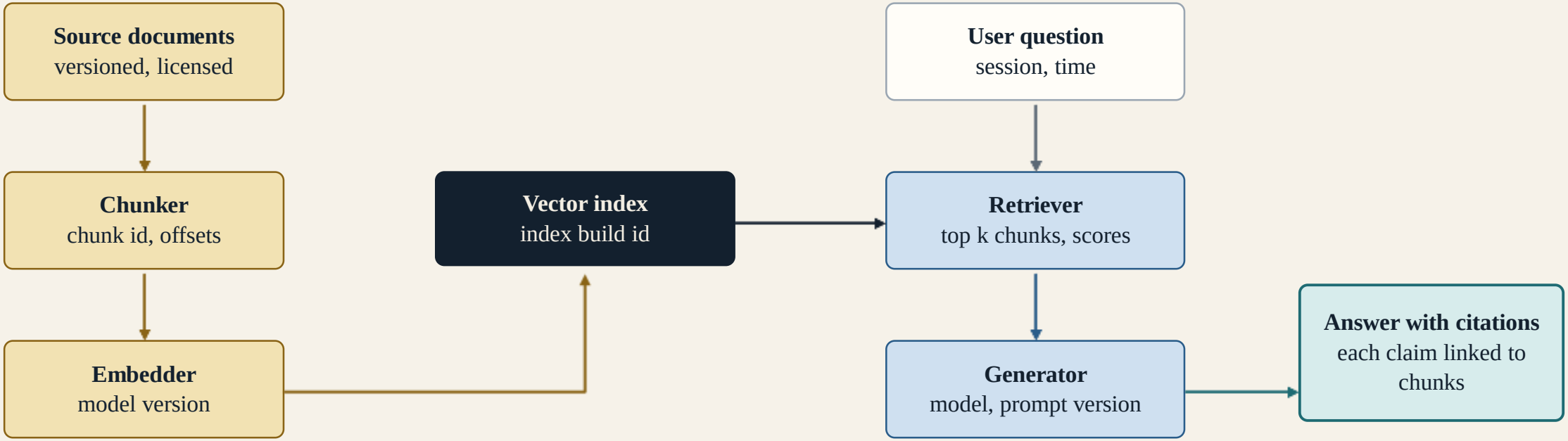
7 Train
Shard order, checkpoints, code, hardware, data seen per step

8 Post train
Preference data origin, annotator pools, reward model lineage

The key artefact is a document level manifest: for every training document, its source, licence status and every filter decision, keyed by content hash.

It enables opt out honouring, contamination audits against benchmarks, copyright response and the public training summaries now required in the EU.

Retrieval augmented generation: provenance per answer



For every answer, record which **document versions** and **chunks** were retrieved, with scores, plus the **model and prompt versions**. Then a wrong answer can be traced to a stale document, a bad chunk boundary, a retrieval miss, or the generator itself.

An answer provenance record, annotated

```
{ "answer_id": "ans_20261106_0042",
"question_hash": "sha256:5d0e…",
"generator": { "model": "[model id]",
"prompt_template": "policy_qa v7" },
"retrieval": { "index_build": "idx_311",
"embedder": "[embedding model id]", "k": 5 },
"citations": [
{ "doc": "leave_policy", "doc_version": "v14",
"chunk": "c0087", "score": 0.83,
"supports": "sentence 2" } ],
"safety_checks": [ "pii_filter v3: pass" ],
"human_feedback": null }
```

Hash the question rather than storing it, where privacy demands.

Version the prompt template like code.

Index build id ties retrieval to a corpus snapshot.

Claim to source mapping says which sentence each chunk supports.

Result: any answer can be replayed and audited months later.

Point in time correctness: provenance against leakage

Feature value known
support tickets = 2

Prediction time
label event at 1 June

Value updated later
support tickets = 9

Correct join: use the latest value with event time before 1 June. The model learns from what was knowable.

Leaky join: use today's value. Offline accuracy looks superb; production collapses.

Feature lineage
Each feature records source tables, transformation code and freshness.

Event timestamps
Every value carries when it became true and when it was recorded.

One definition
Training and serving use the same code, avoiding skew.

Experiment tracking and model registries

Run
params, metrics, code commit, environment

→

Artefacts
model files, plots, dataset references

→

Registry
named versions linked to their run

→

Deployment
endpoint serving a registered version

Tool	Provenance it captures well
MLflow	Runs, parameters, metrics, artefacts, dataset inputs and a model registry with lineage to runs
Weights and Biases	Experiments plus artefact versioning with a lineage graph between datasets and models
DVC	Pipelines and data versions in git; reproducible stages with locked hashes
Kubeflow and Vertex metadata	ML Metadata store recording artefacts, executions and events

Gap to close: these tools often record a dataset path, not its version or hash. Log the snapshot ID or digest, or reproducibility is an illusion.

Label provenance: the humans inside the dataset

Record for every label

- Annotator pseudonym and qualification
- Guideline version in force
- Tool, interface and time spent
- Agreement with other annotators
- Adjudication outcome and adjudicator
- Whether a model pre labelled the item

Why it matters

When a guideline changes midway, labels before and after are inconsistent. Without guideline version provenance, that noise is invisible and unfixable.

Model assisted labels can anchor annotators to the model's errors, so record pre labelling explicitly.

Ethics: provenance should also record working conditions and pay standards for the annotation workforce.

Synthetic data needs more provenance, not less

- **Generator** · model name, version and its own training provenance
- **Recipe** · prompts, seeds, sampling settings, filters
- **Seed data** · which real records conditioned generation
- **Mixing ratio** · share of synthetic data in each training set
- **Privacy tests** · memorisation and re identification checks

Model collapse

Shumailov and colleagues reported in Nature in 2024 that models trained recursively on generated data progressively lose the tails of the original distribution.

Provenance is the only way to know how much of your corpus is already machine made.

And: synthetic is not automatically anonymous. A generator can reproduce records it saw.

Erasure requests meet trained models

1

Request

A person asks for their data to be erased.

2

Locate

Row level lineage finds every copy and derived dataset.

3

Impact

Model lineage lists models trained on those versions.

4

Act

Delete, retrain, or apply and document unlearning.

Design for it

Sharded training, as in SISA style approaches, lets you retrain only the shards that contained the record. That only works if shard membership is recorded.

Evidence it

Record the erasure activity as provenance: what was invalidated, which models were affected and what was done. The invalidation relation from PROV fits exactly.

AI bills of materials: an ingredients list for models

AI BOM for churn_model v3

- **Model** — architecture, weights digest
- **Base model** — name, version, licence
- **Datasets** — ids, versions, licences
- **Software** — libraries and versions
- **Training** — hardware, energy, config
- **Evaluation** — benchmarks, limits

CycloneDX

Added machine learning BOM support in version 1.5 in 2023, with model cards as components.

SPDX 3.0

Introduced AI and Dataset profiles in 2024, alongside its software profiles.

A BOM lists ingredients; an attestation proves the build. Ship both.

SECTION 07 RECAP

Provenance for AI in six moves

1. Instrument eight checkpoints from sourcing to inference.
2. Document with datasheets and cards; generate them from metadata.
3. Treat licence as an inherited, most restrictive property.
4. Keep document level manifests for foundation model corpora.
5. Record answer level provenance for RAG and decisions.
6. Link records to models so erasure and recalls are possible.

Next: the provenance of what we see and hear.

08

SECTION EIGHT

Content provenance

When anyone can generate a photograph, the question “where did this come from?” becomes a question for every citizen.

The authenticity problem

C2PA and Content Credentials

Watermarks, fingerprints and limits

Labelling law and newsroom practice



The authenticity problem has three faces

Synthetic media

Generated images, voices and video of events that never happened, now cheap and photorealistic.

Cheap fakes

Real media with false context: an old photo captioned as today’s, a clip slowed or cut to mislead.

The liar’s dividend

Once fakes are plausible, genuine evidence can be dismissed as fake. A term popularised by Chesney and Citron in 2019.

Detection asks “does this look fake?” It is an arms race that detectors tend to lose as generators improve.

Provenance asks “can this prove where it came from?” It shifts the burden to authentic content carrying evidence.

C2PA: a coalition, a specification and a consumer label

THE COALITION

C2PA

Coalition for Content Provenance and Authenticity, formed in 2021 by Adobe, Arm, BBC, Intel, Microsoft and Truepic. Writes the open standard.

THE COMMUNITY

Content Authenticity Initiative

Adobe led adoption community, reporting more than 6,000 members by January 2026.

THE LABEL

Content Credentials

The public name for a signed C2PA manifest, shown by supporting apps and platforms.

2.2 · May 2025 → **2.3** · January 2026 → **2.4** · April 2026, current → Deployment guidance · July 2026

Adoption spans cameras, phones, editing software, AI image generators and a growing set of social platforms, with a conformance programme to test interoperability between creators, publishers and verifiers.

Anatomy of a C2PA manifest

Image file · photo.jpg

Manifest store · one or more manifests

Assertions
actions taken, ingredients, AI use, hash of content

Claim
lists assertion hashes and the generating tool

Signature
over the claim, with a certificate

Pixel data · bound to the manifest

Hard binding
A cryptographic hash of the content. Any pixel change breaks it.

Soft binding
A watermark or fingerprint that survives resizing and lets a stripped file find its manifest again.

Together these form what C2PA calls **durable** Content Credentials.

A photograph’s journey with Content Credentials

Capture
Device signs manifest 1: camera, time, optional location.

→

Edit
Editor adds manifest 2: crop, exposure; manifest 1 as ingredient.

→

Publish
Newsroom signs manifest 3 and preserves the chain.

→

Share
Platform recompresses; may strip metadata. Soft binding recovers it.

→

Verify
Viewer inspects the chain and each signer.

What the viewer can learn

Signed by a known newsroom · edited in a named application · original captured by a named device model · no generative AI tools reported · two edit steps recorded

What it cannot learn: whether the scene was staged, or whether the caption is honest.

Manifests, watermarks and fingerprints

Property	Signed manifest	Invisible watermark	Fingerprint
Where it lives	Metadata attached to the file	Inside the pixels or audio	In an external database
Information	Rich history, signer, edits	A few bits, an identifier	None in the file
Survives stripping	No	Usually	Yes
Survives heavy edits	Breaks, by design	Partly	Partly
Forgery resistance	Strong, cryptographic	Varies; can be removed or spoofed	Depends on the database
Example	C2PA Content Credentials	SynthID style marks on AI outputs	Perceptual hashing services

No single technique suffices. The robust design layers all three: manifest for richness, watermark to survive stripping, fingerprint to recover the manifest from a lookup.

What Content Credentials can and cannot tell you

Can tell you

- Who signed the record, with which certificate
- Which tools and edits the signers reported
- Whether the file changed since signing
- Whether a generator declared AI involvement

Cannot tell you

- Whether the depicted scene is real or staged
- Anything about files whose credentials were stripped
- That an unsigned file is fake: absence is not evidence
- Whether a trusted signer behaved honestly

The specification itself presents a valid credential as a verified statement about who signed what, not a verdict on truth. An academic security analysis published in 2026 cautioned that the system **should not yet be relied on for high stakes uses** on its own.

Labelling AI content is becoming a legal duty

EUROPEAN UNION

AI Act, Article 50

Providers of generative systems must mark outputs in a machine readable, detectable way; deployers must disclose deepfakes. Transparency duties apply from 2 August 2026; check transitional terms set by the 2026 AI Omnibus.

CHINA

Labelling measures

Rules on labelling AI generated synthetic content took effect on 1 September 2025, requiring both visible labels and embedded metadata.

CALIFORNIA

AI Transparency Act

SB 942 requires large generative AI providers to offer detection tools and embed latent disclosures. Its timing was amended in 2025; confirm the operative date.

Implication for engineers: marking must be designed into generation pipelines, not added at publication. Provenance standards such as C2PA, combined with watermarking, are the practical route to compliance across jurisdictions.

How a verification desk establishes provenance

1 Source
Who first posted it? Contact them; request the original file.

2 Reverse search
Has it appeared before, in another context?

3 Credentials
Inspect metadata and any Content Credentials chain.

4 Geolocate
Match landmarks, signs and terrain to maps.

5 Chronolocate
Shadows, weather records, events: when was it taken?

6 Publish the trail
Explain to readers how it was verified, and sign it.

Notice that only step 3 is cryptographic. Content provenance in journalism is **mostly investigation**, with credentials as one strong signal among several.

SECTION 08 RECAP

Provenance for what we see and hear

1. Provenance inverts the fake detection arms race: authentic media carries evidence.
2. C2PA manifests are signed PROV bundles with ingredients forming a chain.
3. Layer manifests, watermarks and fingerprints to survive real distribution.
4. Credentials prove who stands behind a history, never that a scene is true.
5. Labelling law in the EU, China and California makes marking an engineering requirement.

09

SECTION NINE

Science and reproducibility

Science is the oldest provenance discipline: a result is only knowledge if others can trace and repeat how it was made.

The reproducibility crisis

Workflow systems and notebooks

FAIR principles and RO Crate

ALCOA plus and computational environments



The reproducibility crisis in two numbers

70%

of researchers surveyed had tried and failed to reproduce another scientist’s experiment

50%

had failed to reproduce one of their own experiments

Source: Baker, Nature, 2016, a survey of 1,576 researchers. Figures are “more than” values.

Causes cited include selective reporting, pressure to publish and low statistical power, but also missing methods, code and raw data.

The provenance share of the problem is the part engineering can fix: capture inputs, code, parameters and environment for every result.

Repeat, reproduce, replicate: a ladder of confidence

Repeatable	Same team, same setup, same result. Needs pinned inputs and environment.
Reproducible	Different team, the original data and code, same result. Needs shared, complete provenance.
Replicable	Different team, new data, same conclusion. Needs a clear prospective description of method.
Robust	Conclusion survives reasonable changes in analysis choices. Needs provenance of every choice.

Terminology varies between fields; the ACM, for example, has swapped the meanings of reproducibility and replicability in past badging policies. **Define your terms in every protocol.**

Scientific workflow systems and their provenance

System	Community	Provenance features
Nextflow	Bioinformatics, nf core	Trace and timeline reports per task; container pinning; resumable cached runs
Snakemake	Life sciences, general	Rule graph, file timestamps and hashes, automatic reports
CWL	Cross platform standard	Portable workflow language; CWLProv captures retrospective PROV
Galaxy	Web based life sciences	Full histories of every tool run with parameters, shareable
Common Workflow RO Crates	Packaging layer	Bundles workflow, inputs, outputs and run provenance in one package

Shared pattern: declare steps as functions of inputs, so the engine knows the dependency graph and records it automatically.

Lesson for industry: these communities solved reproducible pipelines years before data platforms did.

Notebooks: powerful, popular and provenance poor

Share of notebooks in a large GitHub study

Ran without errors



24%

Reproduced the same results



4%

Scale for comparison: 100%

Pimentel and colleagues, 2019, analysing about 1.4 million Jupyter notebooks.

Hidden state: cells run out of order leave results no top to bottom run reproduces.

Unpinned dependencies and absolute file paths.

Fixes: restart and run all before saving; lock environments; parameterised execution with tools such as papermill; promote mature notebooks to pipelines.

FAIR principles put provenance at the heart of reuse

F

Findable

Persistent identifiers and rich, indexed metadata.

A

Accessible

Retrievable by identifier through open protocols.

I

Interoperable

Shared vocabularies and qualified references.

R

Reusable

Clear licence and **detailed provenance**.

R1.2 The principle states that metadata should be associated with detailed provenance. It is the only FAIR sub principle devoted to history.

Published by Wilkinson and colleagues in *Scientific Data* in 2016, FAIR is now embedded in research funder policies and increasingly in enterprise data strategies.

RO Crate: packaging research with its provenance

study_2026.crate /

crate metadata file · JSON LD describing everything below

- data / raw and processed files
- workflow / pipeline definition
- results / figures and tables
- run / provenance of execution

People, licences, funders and instruments are described as linked entities using schema.org terms.

Lightweight: a folder plus one metadata file. Works with zip, repositories and git.

Workflow Run profiles capture what ran, with which inputs, producing which outputs.

Enterprise analogue: an audit evidence pack for a regulated report.

ALCOA plus: data integrity in regulated science

A	L	C	O	A
Attributable	**Legible**	**Contemporaneous**	**Original**	**Accurate**
Who did it, and when	Readable for its whole life	Recorded as it happens	The first record, or a certified copy	Error free, edits documented

plus	plus	plus	plus	
Complete	**Consistent**	**Enduring**	**Available**	A complete provenance checklist, written decades before data lakes.
Including repeats and failures	Chronological, ordered	Lasts the retention period	Retrievable for review	

Used by regulators of medicines and medical devices, including the FDA and the UK MHRA, in data integrity guidance for good practice environments.

The computational environment is part of the provenance

Hardware	CPU and GPU model; GPU kernels can be non deterministic
Drivers	CUDA and accelerator library versions
Operating system	Container image pinned by digest, not by tag
Packages	Lock file with exact versions and hashes
Code	Commit hash on a clean working tree
Run settings	Random seeds, parameters, thread counts, time zone

Pin from the bottom up. A tag such as “latest” is a pointer that moves; a digest is a fingerprint that does not.

SECTION 09 RECAP

What science teaches data teams

1. Reproducibility failures are common; much of the cause is missing provenance.
2. Workflow engines capture provenance by construction; notebooks do not.
3. FAIR’s reuse principle requires detailed provenance.
4. ALCOA plus is a ready made integrity checklist.
5. Pin the full environment by digest, from hardware to seeds.

10

SECTION TEN

Regulation and compliance

Behind every rule about data sits a question about where it came from. Regulation is turning provenance from good practice into obligation.

GDPR and the UK regime

EU AI Act Article 10 and the 2026 Omnibus

BCBS 239, life sciences, product rules

Compliance mapping and audit evidence



The regulatory landscape, mapped by what it asks of provenance

Personal data

EU GDPR, UK GDPR and the Data (Use and Access) Act 2025, US state privacy laws

Asks: source, purpose, flows, erasure

Artificial intelligence

EU AI Act and 2026 Omnibus; content labelling rules worldwide

Asks: data origin, governance, logs, marking

Financial services

BCBS 239, SOX, ECB guidance on risk data

Asks: lineage of risk and financial figures

Life sciences

FDA 21 CFR Part 11, EU GMP Annex 11, HIPAA

Asks: audit trails, ALCOA plus

Products and goods

EU deforestation rules, battery passports, FSMA 204

Asks: origin and chain of custody

Software

US secure software attestations, EU Cyber Resilience Act

Asks: SBOMs, build provenance

One capability, many regimes. Build provenance once as a platform service, then map each obligation onto it, rather than building six compliance silos.

GDPR is a provenance law in disguise

Article	Requirement, paraphrased	Provenance capability needed
5(2)	Controller must be able to demonstrate compliance	Evidence of purpose and lawful basis per processing
14 and 15(1)(g)	Tell people the source of data not collected from them	Origin recorded for every personal data element
17 and 19	Erase data and inform recipients	Row level lineage to every copy and recipient
22	Safeguards for solely automated decisions	Decision provenance to explain individual outcomes
30	Records of processing activities	Catalogue of flows, purposes and transfers
33 and 34	Breach notification	Know which data and which people were affected
35	Data protection impact assessment	Map of sources and downstream uses

Purpose limitation is the deepest link: data collected for one purpose must not drift into incompatible ones. Only provenance that records purpose at collection and checks it at each use can enforce that.

Article 10: data governance for high risk AI

Article 10(2) requires training, validation and testing data to be subject to governance practices covering, in paraphrase:

(a)
Relevant design choices

(b)
Collection processes and the **origin of data**; for personal data, its original purpose

(c)
Preparation operations: annotation, labelling, cleaning, updating, enrichment, aggregation

(d)
Assumptions about what the data measures

(e)
Availability, quantity and suitability

(f)
Examination for possible biases

(g)
Measures to detect, prevent and mitigate those biases

(h)
Identification of data gaps and how they are addressed

Points (b) and (c) are provenance by name. Origin plus every preparation step equals a lineage record with rights metadata.

Companions: Article 11 technical documentation and Article 12 automatic logging complete the evidence chain.

The AI Act timeline after the 2026 Omnibus

1 Aug 2024
Act enters into force

2 Feb 2025
Prohibited practices and AI literacy

2 Aug 2025
General purpose AI obligations apply

2 Aug 2026
Article 50 transparency; enforcement powers

24 Jul 2026
AI Omnibus, Regulation 2026/1744, published; in force 27 July

2 Dec 2027
Standalone high risk systems, Annex III, including Article 10

2 Aug 2028
High risk AI in products under Annex I

The Omnibus changed *when* high risk duties apply, not *what* Article 10 requires. Treat the extra time as runway for data provenance work.

General purpose AI: training data provenance goes public

Public training content summary

Providers must publish a sufficiently detailed summary of content used for training, following a template the European Commission issued in July 2025.

Copyright policy

Providers need a policy to comply with EU copyright law, including honouring machine readable text and data mining opt outs.

Technical documentation

Kept for the AI Office and national authorities, including data sources and curation methods.

Downstream information

Integrators receive what they need to meet their own duties, a provenance hand off along the AI supply chain.

Engineering consequence: you cannot write an honest summary, or prove opt outs were honoured, without the document level manifests described in section seven.

BCBS 239: fourteen principles, one recurring weakness

PRINCIPLES 1 TO 2

Governance and infrastructure

Board oversight; integrated data architecture.

PRINCIPLES 3 TO 6

Aggregation capabilities

Accuracy and integrity, completeness, timeliness, adaptability.

PRINCIPLES 7 TO 11

Risk reporting

Accuracy, comprehensiveness, clarity, frequency, distribution.

PRINCIPLES 12 TO 14

Supervisory review

Review, remediation, home and host cooperation.

The lineage gap

Issued in January 2013 with an expected compliance date of 2016 for the largest banks, the principles are still not fully met by most of them. Supervisors repeatedly cite weak data lineage.

Raised expectations

The ECB’s 2024 guide on effective risk data aggregation expects complete, up to date lineage down to attribute level for key risk reports.

Audit trails in health and life sciences

UNITED STATES · SINCE 1997

FDA 21 CFR Part 11

Electronic records need secure, computer generated, time stamped audit trails of operator entries and actions that create, modify or delete them. Changes must not obscure earlier entries.

EUROPEAN UNION

EU GMP Annex 11

Computerised systems in medicines manufacturing need risk based audit trails, reviewed regularly, with reasons for changes.

UNITED STATES

HIPAA

Patients may request an accounting of certain disclosures of their health information, which requires disclosure provenance.

The key phrase: “must not obscure previously recorded information.” That is a regulatory demand for append only, tamper evident provenance, the exact structures of section six.

Product regulations that demand origin evidence

Regulation	Provenance demanded	Status, November 2026
EU Deforestation Regulation	Geolocation of every plot where commodities such as cocoa, coffee, soy and timber were produced	Application postponed twice; confirm current dates
EU Battery Regulation	Digital battery passport with materials origin and carbon footprint	Passport due from February 2027
EU Ecodesign Regulation	Digital product passports across many product groups	Phased by delegated acts
US UFLPA	Prove goods are free of forced labour inputs from a named region	In force since 2022
US FSMA 204	Key data elements at critical tracking events for listed foods	Compliance date extended to 2028
US DSCSA	Electronic, interoperable tracing of prescription drug packages	Phased enforcement

Common thread: the burden of proof has shifted to the seller. Without provenance, goods may simply be refused at the border.

The UK picture: principles, regulators and transparency

UK GDPR and DPA 2018

The same provenance duties as the EU regime: accountability, sources, records of processing, erasure.

Data (Use and Access) Act 2025

Received Royal Assent in June 2025. Adjusts automated decision rules and lawful bases, and creates frameworks for smart data and digital verification services.

Sector regulators for AI

No single AI act. The ICO, FCA, MHRA and others apply existing powers, with the ICO's guidance on AI and data protection central.

Public sector transparency

The Algorithmic Transparency Recording Standard asks government bodies to publish how algorithmic tools work, including data used.

For UK organisations selling into the EU, the EU AI Act still applies to systems placed on the EU market, so provenance must satisfy both regimes.

One capability matrix, many obligations

Obligation	Lineage	Versions	Rights data	Run logs	Signing	Owners
GDPR erasure	Core	Helps	Core	Helps	·	Core
AI Act Art. 10	Core	Core	Core	Helps	Helps	Core
AI Act Art. 12	Helps	Helps	·	Core	Helps	·
BCBS 239	Core	Core	·	Core	Helps	Core
21 CFR Part 11	Helps	Core	·	Core	Core	Core
GPAI summary	Core	Helps	Core	·	·	Helps
Article 50 marking	·	·	Helps	Helps	Core	·
Product passports	Core	Helps	Helps	·	Core	Core

Read across a row to scope a compliance project; read down a column to see how many obligations one capability serves. **Lineage and ownership** pay off almost everywhere. A dot marks little direct relevance.

What goes in a provenance evidence pack

- ✓ Scope: the figure, report or model and its version
- ✓ Lineage export from source to output, at the required depth
- ✓ Run records with input versions and code commits
- ✓ Data quality results at each control point
- ✓ Rights: licences, consents and lawful bases
- ✓ Approvals and change records with named owners
- ✓ Integrity proof: hashes, signatures, log inclusion

Generate, never assemble

If an evidence pack takes weeks of manual work, the provenance system is not finished. The goal is a button.

Package it as a signed, self describing bundle, in the spirit of an RO Crate, so it stays verifiable after export.

SECTION 10 RECAP

Regulation, distilled

1. Nearly every data law contains an unnamed provenance duty.
2. GDPR needs source, purpose and flow evidence per person.
3. AI Act Article 10 names origin and preparation explicitly; deadlines moved, duties did not.
4. BCBS 239 shows that lineage as paperwork fails.
5. Product rules shift the burden of proof to the seller.
6. Build once, map many; generate evidence packs on demand.

Next: supply chains of software, models and goods.

11

SECTION ELEVEN

Supply chains

Software, models and physical goods all travel through chains of strangers. Provenance is how trust survives the journey.



Software supply chain attacks

SLSA, SBOMs and Sigstore

Model supply chain risks

EPCIS and food traceability

Attacks that exploited missing build provenance

2018

Hijacked package

A new maintainer of a popular npm library added a malicious dependency targeting cryptocurrency wallets.

2020

SolarWinds

Attackers inserted code during the build of signed Orion updates; source looked clean.

2021

Codecov

A modified upload script exfiltrated secrets from customers' CI pipelines.

2024

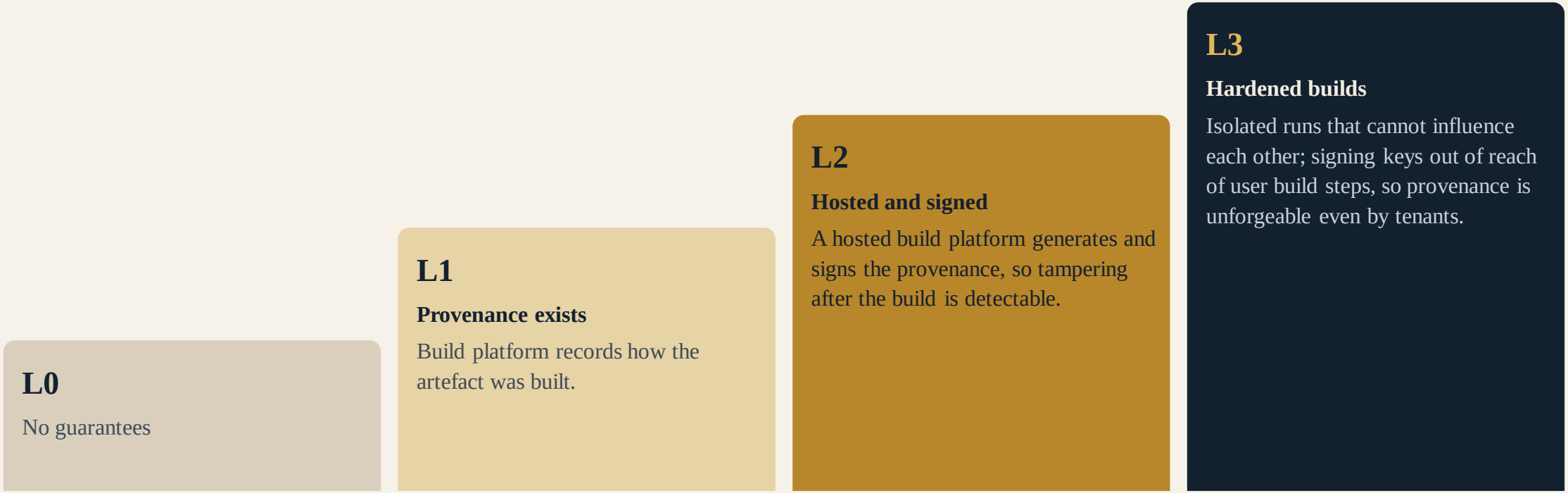
xz utils

A patient, social engineered maintainer hid a backdoor in release tarballs that differed from the repository.

The shared lesson

A signature proves who shipped an artefact, not how it was built. SolarWinds updates were validly signed; the xz tarballs were legitimately published. What was missing was **verifiable build provenance** linking each binary to its exact source and process.

SLSA build levels: a staircase of trust



SLSA, Supply chain Levels for Software Artifacts, is an OpenSSF framework; its version 1.0 in 2023 defined this build track. **The same staircase works for data pipelines.**

Inside a SLSA provenance predicate

```
"predicate": {
"buildDefinition": {
"buildType": "[URI of build template]",
"externalParameters": { "repo", "ref", "workflow" },
"internalParameters": { platform settings },
"resolvedDependencies": [ source commit, base images ]
},
"runDetails": {
"builder": { "id": "[builder identity]" },
"metadata": { "invocationId", "startedOn", "finishedOn" },
"byproducts": [ logs ]
}
}
```

External parameters: what the user controlled. Verifiers check them against policy.

Resolved dependencies: every input, pinned by digest.

Builder id: whom you are trusting to tell the truth.

Replace “build” with “pipeline run” and this is a near perfect data provenance record.

Software bills of materials: knowing what is inside

Minimum elements, US NTIA, 2021

- Supplier name
- Component name
- Version
- Other unique identifiers
- Dependency relationship
- Author of SBOM data
- Timestamp

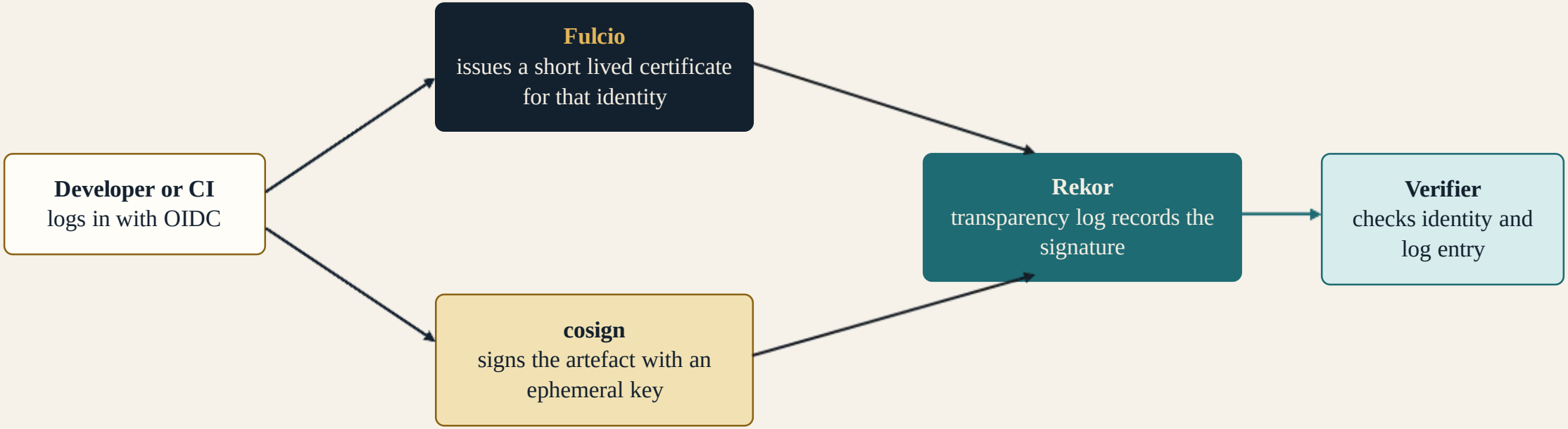
SPDX
Linux Foundation format, an ISO and IEC standard since 2021.

CycloneDX
OWASP format, standardised by Ecma International in 2024.

Policy drivers: US Executive Order 14028 of 2021 and the EU Cyber Resilience Act.

When Log4Shell struck in December 2021, organisations with SBOMs found affected systems in hours; others searched for weeks. **An SBOM is impact analysis for software.**

Sigstore: signing without managing keys



Keys live for minutes and are then discarded; trust rests on **identity plus a public log**. npm package provenance, PyPI attestations and GitHub artefact attestations are built on these components.

The model supply chain: risks and controls

Risk	How it happens	Provenance control
Code in weights files	Pickle based formats can execute code when loaded	Prefer safetensors; scan; verify digest before load
Lookalike models	A typosquatted repository impersonates a popular model	Pin by digest and publisher identity, not by name
Backdoored fine tunes	Hidden triggers inserted during training	Signed training provenance; trusted builders
Poisoned data	Malicious samples planted in scraped sources	Dataset manifests, source reputation, anomaly checks
Silent swaps	A model file replaced after evaluation	Model signing; registry immutability

Model signing is maturing: the OpenSSF model signing specification reached version 1.0 in 2025, using Sigstore so that publishers can sign model files and consumers can verify them before loading.

Farm to fork: one step back, one step forward

Farm
harvest lot, field location

→

Packer
new lot code, cooling record

→

Processor
transformation: many lots in, one out

→

Distributor
shipping and receiving events

→

Retailer
shelf, sale, recall point

Identifiers do the work

GS1 standards give products (GTIN), locations (GLN) and lots their identifiers. A transformation is a derivation: the output lot must reference every input lot.

The weak point

Most firms record only one step up and one step down. Full chain visibility needs shared data, not just shared paper.

Physical provenance is data provenance about atoms: the same entity, activity and agent graph, with lots for entities and handling steps for activities.

EPCIS 2.0: the five dimensions of a physical event

What

Which items, lots or quantities

PROV: entity

When

Event time with time zone

PROV: time

Where

Read point and business location

PROV: location

Why

Business step and disposition

PROV: activity type, role

How

Sensor data such as temperature; new in 2.0

PROV: attributes

Event types

Object (observe, add, delete), aggregation (pack into a pallet), transformation (inputs become outputs), transaction (link to orders), association (attach sensors).

Why it matters here

EPCIS shows that a provenance standard can work across thousands of companies when identifiers and event shapes are agreed in advance. JSON LD support in 2.0 links it to the web of data.

The FDA Food Traceability Rule as a provenance specification

Critical tracking events

- Harvesting
- Cooling
- Initial packing
- First land based receiving
- Shipping
- Receiving
- Transformation

24 h

to supply records to the FDA on request, as an electronic sortable spreadsheet where needed

Key data elements per event, anchored on a **traceability lot code**, for foods on the Food Traceability List.

Compliance date extended by the FDA to July 2028.

SECTION 11 RECAP

Supply chains share one grammar

1. Signatures prove who shipped; build provenance proves how it was made.
2. SLSA levels translate directly to data pipelines.
3. SBOMs list ingredients; attestations prove the recipe ran.
4. Sigstore makes signing routine through identity and logs.
5. Treat models as executable dependencies: pin, verify, sign.
6. EPCIS and FSMA 204 prove provenance works across thousands of firms.

Next: governance, the human system around the technology.

12

SECTION TWELVE

Governance

Technology records history; people decide what must be recorded, who answers for it, and how good is good enough.

Framework, roles and policy

Maturity model and self assessment

Metrics and data contracts

Catalogues and culture



The provenance governance house

Outcome: trusted, lawful, reproducible data and AI

Policy	Standards	Roles	Processes	Metrics
What must be recorded, for which tiers, kept how long	OpenLineage, PROV, naming, facets, signing	Owners, stewards, platform, assurance	Onboarding, change, incident, audit, erasure	Coverage, completeness, time to trace

Foundation: platform capabilities from sections four to six

Every pillar must exist. Tools without policy produce unused graphs; policy without tools produces unenforceable documents.

Who does what: a provenance RACI

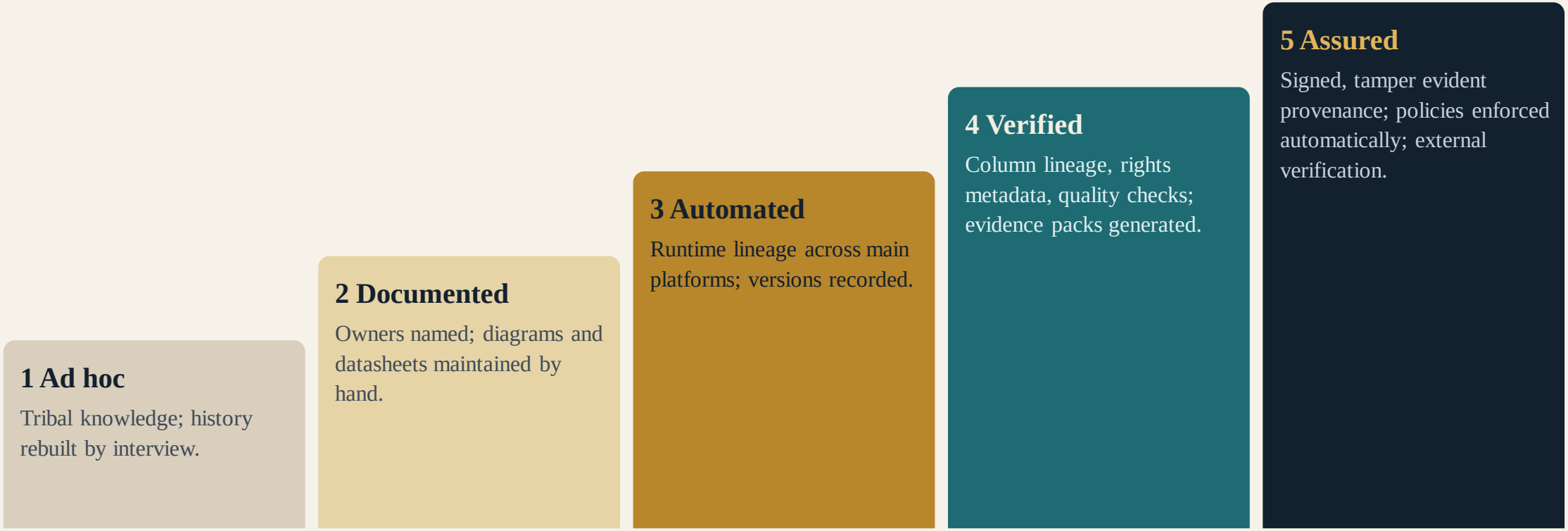
Activity	Data owner	Steward	Engineer	Platform	Risk and legal	ML lead
Set provenance policy and tiers	C	C	I	C	A	C
Record origin and rights	A	R	C	I	C	I
Instrument pipelines	I	I	R	A	I	C
Operate lineage platform	I	I	C	A R	I	I
Review quality of provenance	C	A R	C	C	I	C
Model and dataset cards	C	C	R	I	C	A
Produce audit evidence packs	C	R	C	R	A	C
Respond to erasure requests	A	R	R	C	C	R

R responsible · **A** accountable, exactly one per row · **C** consulted · **I** informed

Model policy clauses you can adapt

P1	Every production dataset shall have a named accountable owner and a recorded origin, collection method and legal basis or licence.
P2	Pipelines producing Tier 1 or Tier 2 assets shall emit run level lineage in the organisation’s standard format, including input versions.
P3	Data shall not be used for a purpose incompatible with its recorded purpose without documented review and approval.
P4	Models released to production shall reference the exact versions and licences of all training and evaluation datasets.
P5	Tier 1 provenance records shall be tamper evident and retained for the audit horizon defined by Risk and Legal.
P6	Provenance records are classified at the level of the data they describe and access is controlled accordingly.

A five level provenance maturity model



Target by tier, not overall. Level 3 for most of the estate and level 4 or 5 for regulated and AI critical assets is a sensible ambition.

Ten questions to place yourself on the model

#	Question	Indicates	Your score
1	Does every production dataset have a named owner?	Level 2	0 1 2
2	Is lineage generated automatically rather than drawn?	Level 3	0 1 2
3	Can you name the input versions any run consumed?	Level 3	0 1 2
4	Do critical reports have column level lineage?	Level 4	0 1 2
5	Are licence and purpose recorded and propagated?	Level 4	0 1 2
6	Can you produce an evidence pack in under a day?	Level 4	0 1 2
7	Can you trace a production model to its training data?	Level 4	0 1 2
8	Are provenance records signed or hash chained?	Level 5	0 1 2
9	Do policies block deployments lacking provenance?	Level 5	0 1 2
10	Can an external party verify your claims independently?	Level 5	0 1 2

Score 0 for no, 1 for partly, 2 for yes. Your level is the highest one where every question scores at least 1.

Programme metrics: from activity to outcome

COVERAGE

Assets with complete provenance

Tier 1 assets meeting the minimum record, as a share

[__ %]

RIGHTS

Datasets with known licence

Including AI training data, as a share

[__ %]

OWNERSHIP

Assets with a named owner

Production scope, as a share

[__ %]

OUTCOME

Mean time to root cause

From incident report to identified source

[__ hours]

OUTCOME

Evidence pack lead time

From audit request to delivery

[__ days]

OUTCOME

Changes with impact check

Schema changes reviewed against lineage first

[__ %]

Gold tiles measure inputs; dark tiles measure what leadership cares about. **Report both**, and baseline them before the programme starts.

Data contracts: provenance promises made in advance



Prospective provenance: the contract states in advance where data comes from and what it means.

Enforced in CI: breaking changes fail the build before they silently drift downstream.

A contract with provenance clauses, simplified

```
dataset: orders
version: 3.1.0
owner: commerce data team
provenance:
origin: checkout service, production
collection: event capture on order placement
legal_basis: contract
allowed_purposes: [fulfilment, finance, fraud]
lineage: openlineage required, run level
schema:
amount: { type: decimal, unit: GBP, nullable: false }
currency: { type: string, values: ISO 4217 }
quality:
row_count_change_max: 20 percent per day
sla:
freshness: 1 hour
```

Allowed purposes make purpose limitation machine checkable.

Units travel with values, the Mars Orbiter lesson applied.

Row count rule would have caught the lost Covid records.

Lineage required turns policy P2 into an executable check.

Surfacing provenance where people choose data

Data catalogue · search results for “revenue”

mart_revenue

Certified · Lineage complete · Licence: internal

Owner: Finance Data · refreshed 2 hours ago

revenue_copy_final

No known producer · No owner

Last modified 14 months ago

Provenance changes behaviour only when it appears **at the moment of choice**.

Badges must be computed from provenance, never granted by hand, or they become decoration.

Rank trusted assets higher in search; let orphans sink and expire.

12 · GOVERNANCE · CULTURE

Make provenance a by product, not a chore

What kills programmes

Forms to fill in after the work is done

Provenance owned only by compliance

No visible benefit to the people capturing it

Blame when lineage reveals a mistake

What sustains them

Capture built into tools people already use

Engineers gain faster debugging first

Leaders ask “where did this come from?” in reviews

Blameless post incident reviews that use lineage

“The best provenance is the provenance nobody had to remember to write.”

SECTION 12 RECAP

Governance that works

1. Five pillars on one platform: policy, standards, roles, processes, metrics.
2. Owners are accountable for origin and rights; platforms for capture.
3. Write testable policy clauses.
4. Target maturity by tier; the weakest link sets your level.
5. Data contracts encode provenance promises as CI checks.
6. Show provenance at the moment of choice; make capture a by product.

Next: a practical roadmap and illustrative cases.

13

SECTION THIRTEEN

Implementation

From ambition to a working capability in twelve months, with illustrative cases showing how the pieces fit.

A twelve month roadmap in three phases

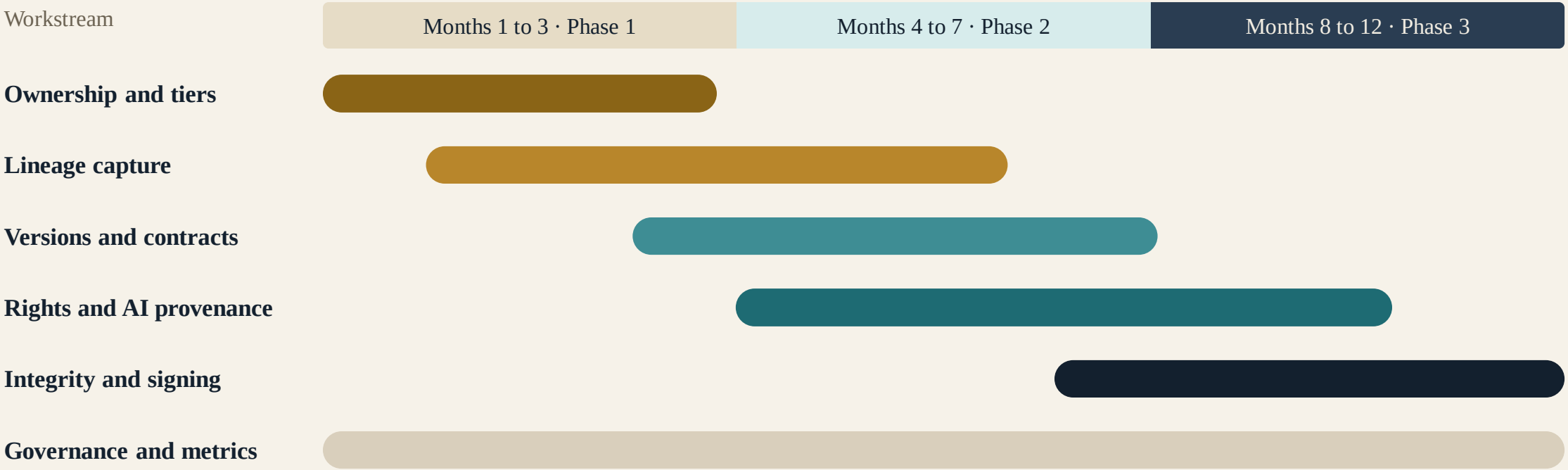
Reference open source stack

Build versus buy

Four illustrative case studies and pitfalls



A twelve month roadmap at a glance



Month 3: first critical report traced end to end

Month 7: production models linked to dataset versions

Month 12: evidence packs generated on demand

Phase 1, months 1 to 3: foundations and a lighthouse

1. **Inventory** critical data assets and reports; assign tiers.
2. **Name owners** for every Tier 1 asset.
3. **Choose standards**: OpenLineage, naming rules, minimum record.
4. **Lighthouse**: trace one painful report end to end.
5. **Baseline** time to root cause and evidence lead time.

Choosing the lighthouse

Pick a report that is visible to executives, has caused a recent incident, and crosses at least three systems. Success there sells the programme.

Exit criteria: owners named, standards agreed, lighthouse traced, baselines measured.

Phase 2, months 4 to 7: automate and extend

Instrument broadly

Enable OpenLineage in orchestrators, Spark and dbt across Tier 1 and Tier 2 pipelines.

Version the data

Adopt open table formats; write snapshot IDs into every run event.

Contracts in CI

Introduce contracts for the most consumed sources; block breaking changes.

Column lineage

For regulated reports and personal data flows first.

ML linkage

Experiment tracking logs dataset digests; model cards generated from runs.

Exit criteria

Coverage target met for Tier 1; production models traceable to dataset versions.

Watch for: naming collisions as more producers emit events. Fix naming at the source rather than with mapping tables.

Phase 3, months 8 to 12: assure and enforce

Rights propagation

Licence and purpose facets flow through lineage; incompatible uses flagged automatically.

Tamper evidence

Hash chain the lineage event log; anchor daily roots externally.

Attestations

Sign provenance for model releases and statutory reports.

Policy as code

Deployment gates refuse Tier 1 releases lacking provenance.

Evidence on demand

One click evidence packs for the top regulated reports.

Exit criteria

Level 4 for Tier 1; outcome metrics improved against baseline.

Then: repeat the cycle for the next tier and platform. Provenance is a capability you operate, not a project you finish.

A reference open source provenance stack

Experience	Catalogue UI: DataHub or OpenMetadata; Marquez UI for lineage exploration
Policy and assurance	Policy engines such as Open Policy Agent; Sigstore and in toto for signing and attestations
Lineage backend	Marquez, or a catalogue ingesting OpenLineage; event log in Kafka or object storage
Capture	OpenLineage integrations for Airflow, Spark, dbt, Flink; sqlglot for static analysis
Versioning	Apache Iceberg or Delta Lake; lakeFS or Nessie; DVC for ML artefacts
ML and content	MLflow, Croissant metadata, CycloneDX or SPDX AI BOMs, C2PA open source SDKs
Quality	Great Expectations or Soda, results emitted as lineage facets

Build, buy or blend?

Criterion	Open source build	Commercial platform	Blend
Time to first value	Slower	Faster	Fast
Coverage of exotic tools	Whatever you build	Vendor connectors	Vendor plus custom emitters
Lock in risk	Low	Higher	Low if OpenLineage is the contract
Operating effort	High	Low	Medium
Custom rights and AI metadata	Full control	Depends on extensibility	Custom facets
Cryptographic assurance	Full control	Often limited	Add signing yourself

Non negotiable in any option: open standards at the boundaries. Insist that a vendor ingests and exports OpenLineage and that you can export your full provenance history.

13 · ILLUSTRATIVE CASE A · BANKING

A bank’s capital report under supervisory scrutiny

Situation

Supervisors question a risk weighted asset figure. Lineage exists only as spreadsheets maintained by consultants. Reconciliation takes weeks.

Intervention

Critical data elements named with owners; runtime lineage across ETL and warehouse; attribute level lineage for the report; bitemporal source tables.

Result

Any figure can be traced to source records as they stood on the reporting date, and evidence packs are generated rather than assembled.

Techniques used: sections 4, 5 and 10. Bitemporality was the hidden hero, because supervisors ask what the bank believed at the time.

Lesson: lineage maintained by people for a regulator decays the day the project ends; generated lineage does not.

Illustrative composite for teaching; not a description of a specific institution.

13 · ILLUSTRATIVE CASE B · HEALTHCARE AI

A triage model whose performance quietly dropped

Situation

A hospital’s triage model underperforms at one site. Nobody can say which data trained the deployed version or whether that site’s patients were represented.

Intervention

Model registry linked to dataset digests; datasheets with site and device composition; inference logs with model version; subgroup evaluation per site.

Result

The cause is traced to a scanner type absent from training data. Retraining is scoped precisely, and the evidence supports regulatory review.

Techniques used: sections 7, 9 and 10. Instrument provenance, the scanner model, is exactly the “how” of the seven questions.

Lesson: representativeness, which Article 10 demands, can only be judged if collection context is recorded.

Illustrative composite for teaching; not a description of a specific organisation.

13 · ILLUSTRATIVE CASE C · MEDIA

A publisher protects trust in its photographs

Situation

A doctored version of the publisher’s photo spreads online with its branding, and a genuine photo is dismissed as fake in the same week.

Intervention

Credential capable cameras for staff; editing tools that preserve manifests; organisational signing at publication; soft binding to survive platform recompression.

Result

Readers and fact checkers can verify which images the publisher stands behind and see their edit history.

Techniques used: sections 6 and 8. The organisation’s certificate and trust list entry became a brand asset.

Lesson: credentials counter the liar’s dividend for your own content; they do not certify anyone else’s.

Illustrative composite for teaching; not a description of a specific publisher.

13 · ILLUSTRATIVE CASE D · PUBLIC SECTOR

A council explains an algorithmic decision

Situation

A resident challenges a benefits prioritisation score. The council must explain which data, model version and rules produced it.

Intervention

Decision provenance records per case; source data with purpose and legal basis; a public transparency record of the tool; human review log.

Result

A clear account is produced within days; an outdated address record is found and corrected; trust is maintained.

Techniques used: sections 7, 10 and 12. Decision provenance served the person affected, not only the institution.

Lesson: in the public sector, provenance is a democratic accountability tool.

Illustrative composite for teaching; not a description of a specific authority.

Eight implementation pitfalls

- ⚠ **Boiling the ocean:** inventorying everything before delivering anything.
- ⚠ **Tool first:** buying a catalogue before naming owners.
- ⚠ **Names without versions:** graphs that cannot answer point in time questions.
- ⚠ **Ignoring rights:** lineage with no licence or purpose metadata.
- ⚠ **Warehouse tunnel vision:** blind to ingestion, BI and ML.
- ⚠ **Premature cryptography:** signing incomplete records.
- ⚠ **No baseline:** unable to prove the programme helped.
- ⚠ **Compliance only framing:** engineers never see a benefit.

SECTION 13 RECAP

From plan to practice

1. Sequence: owners, then automated lineage, then versions, rights and finally integrity.
2. Start with one painful lighthouse report and measure baselines.
3. Blend tools freely, but insist on open standards at every boundary.
4. Across banking, health, media and government, the same patterns recur.
5. Provenance is a capability you operate, not a project you finish.

14

SECTION FOURTEEN

Deeper aspects

What provenance means for knowledge, power and truth, where it reaches its limits, and where it is heading.

Provenance as epistemology

Incompleteness and the horizon

Power, sovereignty and privacy

Agents, the future and open problems



Provenance is how data becomes justified belief

Data are made, not found

Johanna Drucker proposed calling them *capta*, things taken, to stress that every dataset reflects choices about what to capture.

“Raw data” is an oxymoron

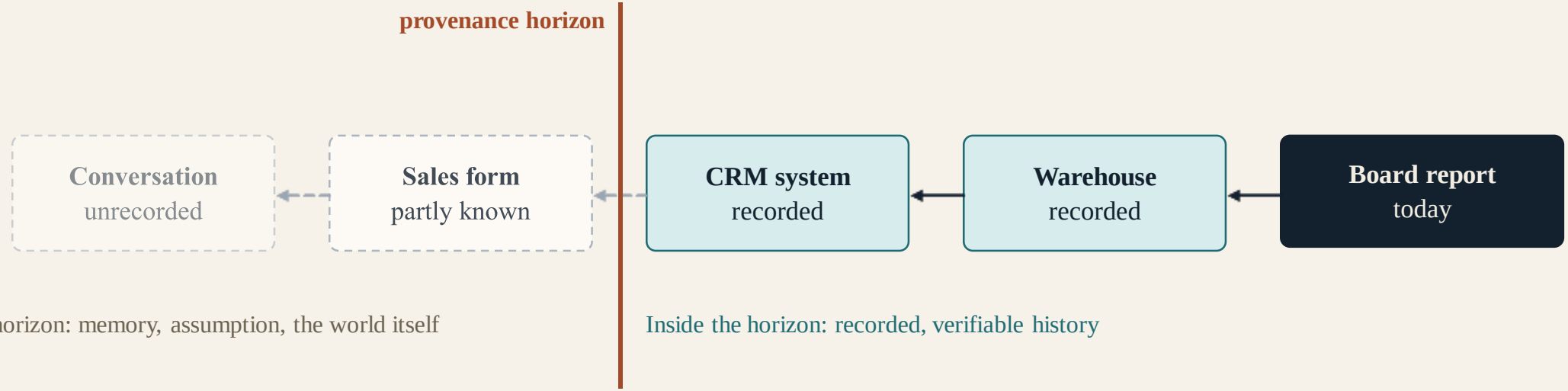
The title of Lisa Gitelman’s 2013 edited volume: data are always already shaped by instruments, categories and purposes.

Testimony

Philosophers of knowledge ask when we may trust what others report. A dataset is testimony from its creators; provenance is the basis for that trust.

If data are the record of decisions, then **provenance is the record of those decisions**, and without it we cannot tell a measurement from an artefact of how it was measured.

Every provenance record has a horizon



Infinite regress: every source has a source. Provenance must stop somewhere, by choice or by necessity.

Honest practice: state where your horizon lies, and what is assumed beyond it.

Whose story does provenance tell?

Who records?

Provenance usually names the organisations that processed data, rarely the people the data describes or the workers who labelled it.

Who can read it?

Institutions see full lineage; data subjects often cannot learn even where their own records came from.

Whose categories?

Classification schemes encode assumptions about identity, place and value that communities may not share.

Indigenous data sovereignty

Indigenous peoples assert rights to govern data about their communities, lands and knowledge, including how its provenance is recorded and shared.

Provenance can entrench power or redistribute it. Designing who appears in the record, and who can query it, is an ethical choice.

Be FAIR and CARE

FAIR · data oriented, 2016

- **F** Findable
- **A** Accessible
- **I** Interoperable
- **R** Reusable, with detailed provenance

CARE · people and purpose oriented, 2019

- **C** Collective benefit
- **A** Authority to control
- **R** Responsibility
- **E** Ethics

The CARE principles, from the Global Indigenous Data Alliance, ask provenance to record not only how data moved but **who has authority over it and whom it should benefit**. Provenance labels, such as the Local Contexts notices, put this into practice.

The provenance and privacy paradox

More provenance
accountability, reproducibility, contestability

Less provenance
minimisation, anonymity, less surveillance



The balance point depends on purpose, risk and who is being recorded.

Pseudonymise agents in provenance shown widely; keep the key restricted.

Aggregate provenance for statistics; keep detail for audits only.

Prove without revealing: hashes, selective disclosure, zero knowledge techniques.

14 · DEEPER ASPECTS · A WARNING

A perfectly traced lie is still a lie.

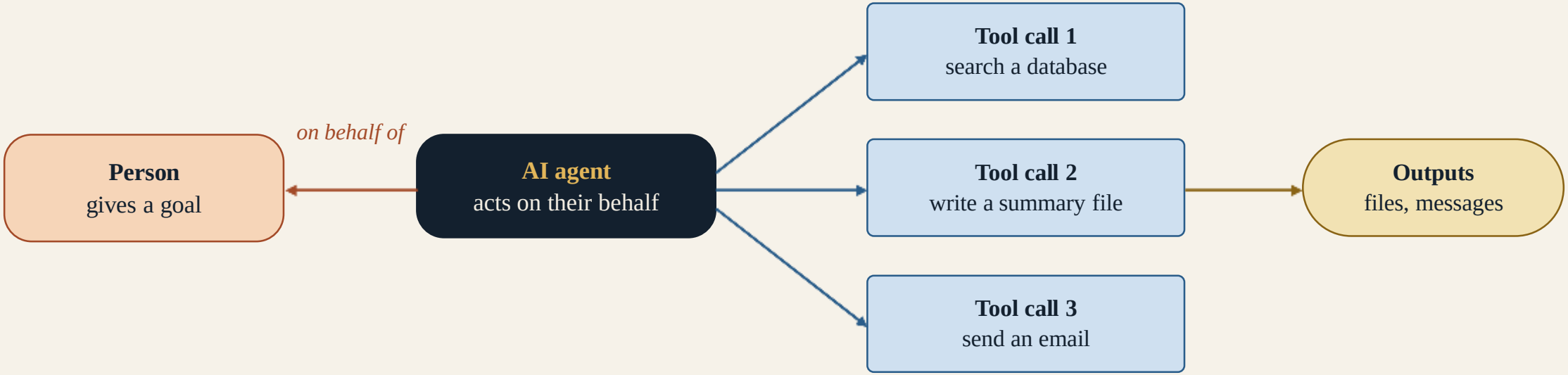
Provenance tells you where something came from and who stands behind it.

Quality tells you whether it is accurate, complete and consistent.

Context tells you whether it is fit for this purpose.

Trust needs all three. Provenance makes the other two checkable; it does not replace them.

When agents act: provenance of autonomous work



Delegation is PROV's actedOnBehalfOf: record the human principal and the scope granted.

Every tool call is an activity with inputs, outputs, model version and prompt.

Outputs should carry markers that an agent, not a person, produced them.

Six trends shaping the next five years

Convergence

OpenLineage, PROV, C2PA, SLSA and the D&TA fields move towards interoperable mappings.

Provenance by default

Engines, cameras and AI tools emit provenance without configuration.

Policy as code

Rights and purpose checks executed automatically at every use.

Confidential provenance

Enclaves and zero knowledge proofs attest without exposing data.

Agent accountability

Delegation chains for AI agents become audit requirements.

Post quantum signing

Long lived provenance migrates to quantum resistant signatures.

These are informed expectations, not predictions with certainty. Watch the standards bodies named throughout this masterclass for signals.

Provenance must outlive today’s cryptography

The risk

A large quantum computer could forge signatures made with today’s RSA and elliptic curve schemes. Provenance kept for decades, such as clinical, legal or archival records, could then be backdated or falsified.

The response

NIST published post quantum standards in August 2024, including two signature schemes: a lattice based scheme in FIPS 204 and a hash based scheme in FIPS 205.

Practical steps now

- Inventory where provenance signatures must last beyond ten years
- Design for algorithm agility in record formats
- Use trusted timestamps and periodically re anchor archives
- Hashes such as SHA 256 remain comparatively robust

Open problems worth a career

Problem	Why it is hard
Training data attribution	Which training examples most influenced a model’s output? Exact answers are expensive at scale.
Proving absence	Demonstrating that a document was not used, or was truly unlearned, is far harder than proving use.
Semantic bridging	Automatically linking low level system traces to business meaning.
Cross organisation lineage	Joining provenance across companies without revealing confidential structure.
Usable provenance	Explaining complex histories to non experts without oversimplifying.
Robust content binding	Credentials that survive heavy edits yet resist forgery and stripping.
Provenance of reasoning	Faithfully recording why an AI system reached a conclusion, not just what it consumed.

Each is an active research area spanning databases, security, machine learning and human computer interaction.

SECTION 14 RECAP

The deeper lessons

1. Data are made; provenance records how, which is the basis of justified belief.
2. Every record has a horizon; declare it.
3. Provenance distributes power; design who is recorded and who may read.
4. Balance accountability with minimisation; prove without revealing.
5. Provenance is not truth; pair it with quality and context.
6. Agents and quantum computing set the next agenda.

Finally: synthesis, glossary, references and exercises.

15

SECTION FIFTEEN

Synthesis and resources

Twelve principles to remember, a practitioner checklist, a glossary, references and exercises to consolidate learning.

Twelve principles of provenance

Practitioner checklist

Glossary and references

Exercises and close



15 · SYNTHESIS

Twelve principles of data provenance

1

Verifiable or it is a story. Evidence beats narrative.

2

Capture as a by product, never as paperwork.

3

Versions, not names. Point at exact states.

4

Terms travel with data. Licence and purpose propagate.

5

Every step has an owner. Accountability is a field.

6

Tier by risk. Depth follows stakes.

7

Open standards at boundaries. History outlives tools.

8

Evidence immutable, views disposable.

9

Protect provenance like the data it describes.

10

Declare the horizon. Admit what is unrecorded.

11

Provenance is not truth. Pair with quality and context.

12

Serve the people recorded, not only the institution.

Gold: engineering. Teal: operating model. Dark: ethics and epistemics. Together they summarise all fifteen sections.

The practitioner’s provenance checklist

For every dataset

- ☐ Named owner and steward
- ☐ Origin, method and date recorded
- ☐ Licence, purpose and legal basis
- ☐ Versioned with immutable identifiers
- ☐ Datasheet or card kept with each version
- ☐ Retention aligned with audit horizon

For every pipeline, model and release

- ☐ Run events with input versions
- ☐ Code commit, parameters and environment
- ☐ Row counts and quality results per hop
- ☐ Column lineage for critical outputs
- ☐ Models linked to dataset digests
- ☐ Signed provenance for Tier 1 releases

Glossary, part one

Activity. In PROV, something that occurs over time and acts on entities.

Agent. A person, organisation or software bearing responsibility for an activity.

Attestation. A signed, typed claim about an artefact identified by digest.

Bitemporal data. Records carrying both valid time and transaction time.

Bundle. A named set of provenance statements, itself an entity.

C2PA. Coalition for Content Provenance and Authenticity and its specification.

Chain of custody. Unbroken record of who held an item and when.

Column lineage. Dependencies between individual fields, direct or indirect.

Content addressing. Naming data by the hash of its bytes.

Content Credentials. Public name for a signed C2PA manifest.

Data contract. Producer commitment on schema, meaning, origin and service levels.

Derivation. The claim that one entity's content was transformed from another.

Entity. In PROV, a thing with fixed aspects: data, file, document, object.

Facet. In OpenLineage, a typed extension attached to a job, run or dataset.

Glossary, part two

Hard binding. A cryptographic hash tying a manifest to exact content.

Merkle tree. Hash tree whose root commits to every leaf.

Pedigree. The reliability and authority of a dataset's ancestors.

Provenance horizon. The point beyond which history is not recorded.

Retrospective provenance. The record of what actually ran.

SLSA. Graduated framework for build provenance and integrity.

Transparency log. Public, append only, verifiable log of entries.

Lineage. The dependency graph of how data flowed between assets.

OpenLineage. Open standard for run, job and dataset lineage events.

PROV. The W3C family of provenance standards published in 2013.

Provenance semiring. Algebra annotating query results with how they were derived.

SBOM. Software bill of materials listing components and relationships.

Soft binding. Watermark or fingerprint linking content to its manifest.

Why provenance. The sets of input tuples that justify an output.

Key research works

Cui, Widom and Wiener (2000). Tracing the lineage of view data in a warehousing environment. *ACM TODS*.

Green, Karvounarakis and Tannen (2007). Provenance Semirings. *PODS*.

Moreau et al. (2011). The Open Provenance Model core specification. *Future Generation Computer Systems*.

Baker (2016). 1,500 scientists lift the lid on reproducibility. *Nature*.

Mitchell et al. (2019). Model Cards for Model Reporting. *FAT* Conference*.

Carroll et al. (2020). The CARE Principles for Indigenous Data Governance. *Data Science Journal*.

Longpre et al. (2023). The Data Provenance Initiative: an audit of dataset licensing and attribution in AI. *arXiv*.

Herndon, Ash and Pollin (2014). Does high public debt consistently stifle economic growth? *Cambridge Journal of Economics*.

Buneman, Khanna and Tan (2001). Why and Where: A Characterization of Data Provenance. *ICDT*.

Chapman and Jagadish (2009). Why Not? *SIGMOD*.

Wilkinson et al. (2016). The FAIR Guiding Principles. *Scientific Data*.

Ziemann, Eren and El Osta (2016). Gene name errors are widespread in the scientific literature. *Genome Biology*.

Pimentel et al. (2019). A large scale study about quality and reproducibility of Jupyter notebooks. *MSR*.

Gebru et al. (2021). Datasheets for Datasets. *Communications of the ACM*.

Shumailov et al. (2024). AI models collapse when trained on recursively generated data. *Nature*.

Gitelman, ed. (2013). “Raw Data” Is an Oxymoron. *MIT Press*.

Standards, specifications and laws

W3C PROV family, 2013: PROV DM, PROV O, PROV N, PROV CONSTRAINTS.

C2PA Technical Specification 2.4, April 2026; Deployment Guidance 1.0, July 2026.

SPDX, ISO and IEC 5962:2021; **CycloneDX**, Ecma 424.

Data and Trust Alliance Data Provenance Standards 1.0.0, 2024; OASIS DPS Technical Committee.

MLCommons Croissant; **RO Crate** specification.

GDPR, Regulation (EU) 2016/679; UK GDPR; Data (Use and Access) Act 2025.

BCBS 239, Basel Committee, 2013; ECB guide on risk data aggregation, 2024.

NIST FIPS 204 and 205, post quantum signature standards, 2024.

OpenLineage specification and Marquez, LF AI and Data Foundation.

SLSA 1.0, OpenSSF, 2023; **in toto** Attestation Framework.

RFC 6962 Certificate Transparency; **RFC 3161** Time Stamp Protocol.

W3C Verifiable Credentials Data Model; DCAT 3; ODRL.

GS1 EPCIS 2.0, 2022; FDA Food Traceability Rule, FSMA section 204.

EU AI Act, Regulation (EU) 2024/1689, as amended by Regulation (EU) 2026/1744.

FDA 21 CFR Part 11; EU GMP Annex 11.

NTIA Minimum Elements for a Software Bill of Materials, 2021.

15 · EXERCISES

Six exercises, one per learning outcome

DEFINE

Explain to a non specialist why your lineage tool is not yet a provenance system. Use the seven questions.

MODEL

Write one real pipeline in PROV N; identify every derivation that needs judgement.

ENGINEER

Emit OpenLineage events from one job into Marquez, including input versions and row counts.

SECURE

Hash chain a provenance log, then edit an entry and show that verification fails.

COMPLY

Map Article 10(2) points (b) and (c) to fields your current systems hold, and list gaps.

REFLECT

Identify one dataset’s provenance horizon and one group whose agency it omits.

Capstone: produce a one page evidence pack for a single important figure in your organisation, from source to dashboard, and time how long it takes.

CLOSING THOUGHT

Behind every record are people: those who made it, and those it describes.

Provenance is how we keep faith with both. Record honestly, protect carefully, and always be able to answer the question: *where did this come from?*







Illustrative portraits